
COMMUNICATION

Interprocess communication is at the heart of all distributed systems. It makes no sense to study distributed systems without carefully examining the ways that processes on different machines can exchange information. Communication in distributed systems has traditionally always been based on low-level message passing as offered by the underlying network. Expressing communication through message passing is harder than using primitives based on shared memory, as available for nondistributed platforms. Modern distributed systems often consist of thousands or even millions of processes scattered across a network with unreliable communication such as the Internet. Unless the primitive communication facilities of computer networks are replaced by something else, development of large-scale distributed applications is extremely difficult.

In this chapter, we start by discussing the rules that communicating processes must adhere to, known as protocols, and concentrate on structuring those protocols in the form of layers. We then look at two widely-used models for communication: Remote Procedure Call (RPC), and Message-Oriented Middleware (MOM). We also discuss the general problem of sending data to multiple receivers, called multicasting.

Our first model for communication in distributed systems is the remote procedure call (RPC). An RPC aims at hiding most of the intricacies of message passing, and is ideal for client-server applications. However, realizing RPCs in a transparent manner is easier said than done. We look at a number of important details that cannot be ignored, while diving into actual code to illustrate to what extent distribution transparency can be realized such that performance is still acceptable.

In many distributed applications, communication does not follow the rather strict pattern of client-server interaction. In those cases, it turns out that thinking in terms of messages is more appropriate. The low-level communication facilities of computer networks are in many ways not suitable,

again due to their lack of distribution transparency. An alternative is to use a high-level message-queuing model, in which communication proceeds much the same as in e-mail systems. Message-oriented communication is a subject important enough to warrant a section of its own. We look at numerous aspects, including application-level routing.

Finally, since our understanding of setting up multicast facilities has improved, novel and elegant solutions for data dissemination have emerged. We pay separate attention to this subject in the last section of this chapter, discussing traditional deterministic means of multicasting, as well as probabilistic approaches as used in flooding and gossiping. The latter have been receiving increased attention over the past years due to their elegance and simplicity.

4.1 Foundations

Before we start our discussion on communication in distributed systems, we first recapitulate some of the fundamental issues related to communication. In the next section we briefly discuss network communication protocols, as these form the basis for any distributed system. After that, we take a different approach by classifying the different types of communication that usually occur in distributed systems.

Layered Protocols

Due to the absence of shared memory, all communication in distributed systems is based on sending and receiving (low level) messages. When process P wants to communicate with process Q, it first builds a message in its own address space. Then it executes a system call that causes the operating system to send the message over the network to Q. Although this basic idea sounds simple enough, in order to prevent chaos, P and Q have to agree on the meaning of the bits being sent.

The OSI reference model

To make it easier to deal with the numerous levels and issues involved in communication, the International Standards Organization (ISO) developed a reference model that clearly identifies the various levels involved, gives them standard names, and points out which level should do which job. This model is called the **Open Systems Interconnection Reference Model** [Day and Zimmerman, 1983] usually abbreviated as **ISO OSI** or sometimes just the **OSI model**. It should be emphasized that the protocols that were developed as part of the OSI model were never widely used and are essentially dead. However, the underlying model itself has proved to be quite useful for understanding computer networks. Although we do not intend to give a full

description of this model and all of its implications here, a short introduction will be helpful. For more details see [Tanenbaum and Wetherall, 2010].

The OSI model is designed to allow open systems to communicate. An open system is one that is prepared to communicate with any other open system by using standard rules that govern the format, contents, and meaning of the messages sent and received. These rules are formalized in what are called **communication protocols**. To allow a group of computers to communicate over a network, they must all agree on the protocols to be used. A protocol is said to provide a **communication service**. There are two types of such services. In the case of a **connection-oriented** service, before exchanging data the sender and receiver first explicitly establish a connection, and possibly negotiate specific parameters of the protocol they will use. When they are done, they release (terminate) the connection. The telephone is a typical connection-oriented communication service. With **connectionless services**, no setup in advance is needed. The sender just transmits the first message when it is ready. Dropping a letter in a mailbox is an example of making use of connectionless communication service. With computers, both connection-oriented and connectionless communication are common.

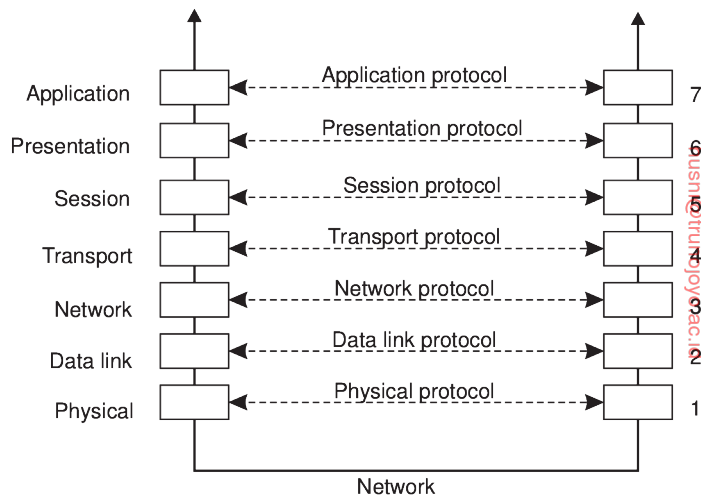


Figure 4.1: Layers, interfaces, and protocols in the OSI model.

In the OSI model, communication is divided into seven levels or layers, as shown in Figure 4.1. Each layer offers one or more specific communication services to the layer above it. In this way, the problem of getting a message from A to B can be divided into manageable pieces, each of which can be solved independently of the others. Each layer provides an **interface** to the one above it. The interface consists of a set of operations that together define the service the layer is prepared to offer. The seven OSI layers are:

Physical layer Deals with standardizing how two computers are connected and how 0s and 1s are represented.

Data link layer Provides the means to detect and possibly correct transmission errors, as well as protocols to keep a sender and receiver in the same pace.

Network layer Contains the protocols for routing a message through a computer network, as well as protocols for handling congestion.

Transport layer Mainly contains protocols for directly supporting applications, such as those that establish reliable communication, or support real-time streaming of data.

Session layer Provides support for sessions between applications.

Presentation layer Prescribes how data is represented in a way that is independent of the hosts on which communicating applications are running.

Application layer Essentially, everything else: e-mail protocols, Web access protocols, file-transfer protocols, and so on.

When process P wants to communicate with some remote process Q, it builds a message and passes that message to the application layer as offered to it by means of an interface. This interface will typically appear in the form of a library procedure. The application layer software then adds a **header** to the front of the message and passes the resulting message across the layer 6/7 interface to the presentation layer. The presentation layer, in turn, adds its own header and passes the result down to the session layer, and so on. Some layers add not only a header to the front, but also a trailer to the end. When it hits the bottom, the physical layer actually transmits the message (which by now might look as shown in Figure 4.2) by putting it onto the physical transmission medium.

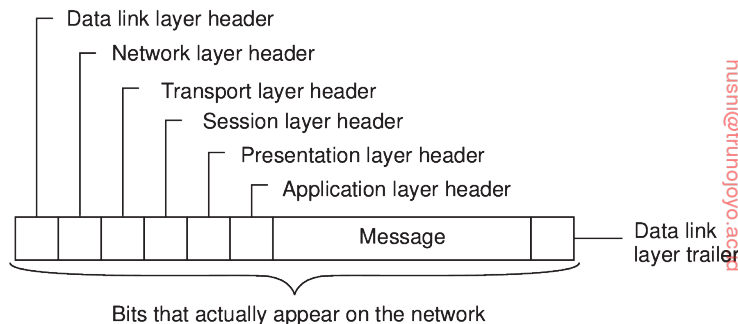


Figure 4.2: A typical message as it appears on the network.

When the message arrives at the remote machine hosting Q, it is passed upward, with each layer stripping off and examining its own header. Finally, the message arrives at the receiver, process Q, which may reply to it using

the reverse path. The information in the layer- n header is used for the layer- n protocol.

In the OSI model, there are not two layers, but seven, as we saw in Figure 4.1. The collection of protocols used in a particular system is called a **protocol suite** or **protocol stack**. It is important to distinguish a *reference model* from its actual *protocols*. As said, the OSI protocols were never popular, in contrast to protocols developed for the Internet, such as TCP and IP.

Note 4.1 (More information: Protocols in the OSI model)

Let us briefly examine each of the OSI layers in turn, starting at the bottom. Instead of giving examples of OSI protocols, where appropriate, we will point out some of the Internet protocols used in each layer.

Lower-level protocols. The three lowest layers of the OSI protocol suite implement the basic functions that encompass a computer network.

The physical layer is concerned with transmitting the 0s and 1s. How many volts to use for 0 and 1, how many bits per second can be sent, and whether transmission can take place in both directions simultaneously are key issues in the physical layer. In addition, the size and shape of the network connector (plug), as well as the number of pins and meaning of each are of concern here.

The physical layer protocol deals with standardizing the electrical, optical, mechanical, and signaling interfaces so that when one machine sends a 0 bit it is actually received as a 0 bit and not a 1 bit. Many physical layer standards have been developed (for different media), for example, the USB standard for serial communication lines.

The physical layer just sends bits. As long as no errors occur, all is well. However, real communication networks are subject to errors, so some mechanism is needed to detect and correct them. This mechanism is the main task of the data link layer. What it does is to group the bits into units, sometimes called **frames**, and see that each frame is correctly received.

The data link layer does its work by putting a special bit pattern on the start and end of each frame to mark them, as well as computing a **checksum** by adding up all the bytes in the frame in a certain way. The data link layer appends the checksum to the frame. When the frame arrives, the receiver recomputes the checksum from the data and compares the result to the checksum following the frame. If the two agree, the frame is considered correct and is accepted. If they disagree, the receiver asks the sender to retransmit it. Frames are assigned sequence numbers (in the header), so everyone can tell which is which.

On a LAN, there is usually no need for the sender to locate the receiver. It just puts the message out on the network and the receiver takes it off. A wide-area network, however, consists of a large number of machines, each with some number of lines to other machines, rather like a large-scale map showing major cities and roads connecting them. For a message to get from the sender to the receiver it may have to make a number of hops, at each one choosing an outgoing line to use.

The question of how to choose the best path is called **routing**, and is essentially the primary task of the network layer.

The problem is complicated by the fact that the shortest route is not always the best route. What really matters is the amount of delay on a given route, which, in turn, is related to the amount of traffic and the number of messages queued up for transmission over the various lines. The delay can thus change over the course of time. Some routing algorithms try to adapt to changing loads, whereas others are content to make decisions based on long-term averages.

At present, the most widely used network protocol is the connectionless **IP (Internet Protocol)**, which is part of the Internet protocol suite. An IP **packet** (the technical term for a message in the network layer) can be sent without any setup. Each IP packet is routed to its destination independent of all others. No internal path is selected and remembered.

Transport protocols. The transport layer forms the last part of what could be called a basic network protocol stack, in the sense that it implements all those services that are not provided at the interface of the network layer, but which are reasonably needed to build network applications. In other words, the transport layer turns the underlying network into something that an application developer can use.

Packets can be lost on the way from the sender to the receiver. Although some applications can handle their own error recovery, others prefer a reliable connection. The job of the transport layer is to provide this service. The idea is that the application layer should be able to deliver a message to the transport layer with the expectation that it will be delivered without loss.

Upon receiving a message from the application layer, the transport layer breaks it into pieces small enough for transmission, assigns each one a sequence number, and then sends them all. The discussion in the transport layer header concerns which packets have been sent, which have been received, how many more the receiver has room to accept, which should be retransmitted, and similar topics.

Reliable transport connections (which by definition are connection-oriented) can be built on top of connection-oriented or connectionless network services. In the former case all the packets will arrive in the correct sequence (if they arrive at all), but in the latter case it is possible for one packet to take a different route and arrive earlier than the packet sent before it. It is up to the transport layer software to put everything back in order to maintain the illusion that a transport connection is like a big tube—you put messages into it and they come out undamaged and in the same order in which they went in. Providing this end-to-end communication behavior is an important aspect of the transport layer.

The Internet transport protocol is called **TCP (Transmission Control Protocol)** and is described in detail by Comer [2013]. The combination TCP/IP is now used as a de facto standard for network communication. The Internet protocol suite also supports a connectionless transport protocol called **UDP (Universal Datagram**

Protocol), which is essentially just IP with some minor additions. User programs that do not need a connection-oriented protocol normally use UDP.

Additional transport protocols are regularly proposed. For example, to support real-time data transfer, the **Real-time Transport Protocol (RTP)** has been defined. RTP is a framework protocol in the sense that it specifies packet formats for real-time data without providing the actual mechanisms for guaranteeing data delivery. In addition, it specifies a protocol for monitoring and controlling data transfer of RTP packets [Schulzrinne et al., 2003]. Likewise, the **Streaming Control Transmission Protocol (SCTP)** has been proposed as an alternative to TCP [Stewart, 2007]. The main difference between SCTP and TCP is that SCTP groups data into messages, whereas TCP merely moves bytes between processes. Doing so may simplify application development.

Higher-level protocols. Above the transport layer, OSI distinguishes three additional layers. In practice, only the application layer is ever used. In fact, in the Internet protocol suite, everything above the transport layer is grouped together. In the face of middleware systems, we shall see that neither the OSI nor the Internet approach is really appropriate.

The session layer is essentially an enhanced version of the transport layer. It provides dialog control, to keep track of which party is currently talking, and it provides synchronization facilities. The latter are useful to allow users to insert checkpoints into long transfers, so that in the event of a crash, it is necessary to go back only to the last checkpoint, rather than all the way back to the beginning. In practice, few applications are interested in the session layer and it is rarely supported. It is not even present in the Internet protocol suite. However, in the context of developing middleware solutions, the concept of a session and its related protocols has turned out to be quite relevant, notably when defining higher-level communication protocols.

Unlike the lower layers, which are concerned with getting the bits from the sender to the receiver reliably and efficiently, the presentation layer is concerned with the meaning of the bits. Most messages do not consist of random bit strings, but more structured information such as people's names, addresses, amounts of money, and so on. In the presentation layer it is possible to define records containing fields like these and then have the sender notify the receiver that a message contains a particular record in a certain format. This makes it easier for machines with different internal representations to communicate with each other.

The OSI application layer was originally intended to contain a collection of standard network applications such as those for electronic mail, file transfer, and terminal emulation. By now, it has become the container for all applications and protocols that in one way or the other do not fit into one of the underlying layers. From the perspective of the OSI reference model, virtually all distributed systems are just applications.

What is missing in this model is a clear distinction between applications, application-specific protocols, and general-purpose protocols. For example, the Internet **File Transfer Protocol (FTP)** [Postel and Reynolds, 1985; Horowitz and

Lunt, 1997] defines a protocol for transferring files between a client and server machine. The protocol should not be confused with the ftp program, which is an end-user application for transferring files and which also (not entirely by coincidence) happens to implement the Internet FTP.

Another example of a typical application-specific protocol is the **HyperText Transfer Protocol (HTTP)** [Fielding et al., 1999] which is designed to remotely manage and handle the transfer of Web pages. The protocol is implemented by applications such as Web browsers and Web servers. However, HTTP is now also used by systems that are not intrinsically tied to the Web. For example, Java's object-invocation mechanism can use HTTP to request the invocation of remote objects that are protected by a firewall [Oracle, 2010].

There are also many general-purpose protocols that are useful to many applications, but which cannot be qualified as transport protocols. In many cases, such protocols fall into the category of middleware protocols.

Middleware protocols

Middleware is an application that logically lives (mostly) in the OSI application layer, but which contains many general-purpose protocols that warrant their own layers, independent of other, more specific applications. Let us briefly look at some examples.

The **Domain Name System (DNS)** [Liu and Albitz, 2006] is a distributed service that is used to look up a network address associated with a name, such as the address of a so-called **domain name** like `www.distributed-systems.net`. In terms of the OSI reference model, DNS is an application and therefore is logically placed in the application layer. However, it should be quite obvious that DNS is offering a general-purpose, application-independent service. Arguably, it forms part of the middleware.

As another example, there are various ways to establish authentication, that is, provide proof of a claimed identity. Authentication protocols are not closely tied to any specific application, but instead, can be integrated into a middleware system as a general service. Likewise, authorization protocols by which authenticated users and processes are granted access only to those resources for which they have authorization, tend to have a general, application-independent nature. Being labeled as applications in the OSI reference model, these are clear examples that belong in the middleware.

Distributed commit protocols establish that in a group of processes, possibly spread out across a number of machines, either all processes carry out a particular operation, or that the operation is not carried out at all. This phenomenon is also referred to as **atomicity** and is widely applied in transactions. As it turns out, commit protocols can present an interface independently of specific applications, thus providing a general-purpose transaction service.

In such a form, they typically belong to the middleware and not to the OSI application layer.

As a last example, consider a distributed locking protocol by which a resource can be protected against simultaneous access by a collection of processes that are distributed across multiple machines. It is not hard to imagine that such protocols can be designed in an application-independent fashion, and accessible through a relatively simple, again application-independent interface. As such, they generally belong in the middleware.

These protocol examples are not directly tied to communication, yet there are also many middleware communication protocols. For example, with a so-called **remote procedure call**, a process is offered a facility to *locally* call a procedure that is effectively implemented on a *remote* machine. This communication service belongs to one of the oldest types of middleware services and is used for realizing access transparency. In a similar vein, there are high-level communication services for setting and synchronizing streams for transferring real-time data, such as needed for multimedia applications. As a last example, some middleware systems offer reliable multicast services that scale to thousands of receivers spread across a wide-area network.

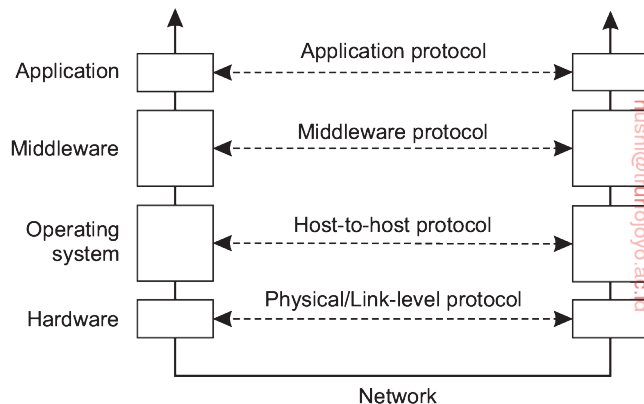


Figure 4.3: An adapted reference model for networked communication.

Taking this approach to layering leads to the adapted and simplified reference model for communication, as shown in Figure 4.3. Compared to the OSI model, the session and presentation layer have been replaced by a single middleware layer that contains application-independent protocols. These protocols do not belong in the lower layers we just discussed. Network and transport services have been grouped into communication services as normally offered by an operating system, which, in turn, manages the specific lowest-level hardware used to establish communication.

Types of Communication

In the remainder of this chapter, we concentrate on high-level middleware communication services. Before doing so, there are other general criteria for distinguishing (middleware) communication. To understand the various alternatives in communication that middleware can offer to applications, we view the middleware as an additional service in client-server computing, as shown in Figure 4.4. Consider, for example an electronic mail system. In principle, the core of the mail delivery system can be seen as a middleware communication service. Each host runs a user agent allowing users to compose, send, and receive e-mail. A sending user agent passes such mail to the mail delivery system, expecting it, in turn, to eventually deliver the mail to the intended recipient. Likewise, the user agent at the receiver's side connects to the mail delivery system to see whether any mail has come in. If so, the messages are transferred to the user agent so that they can be displayed and read by the user.

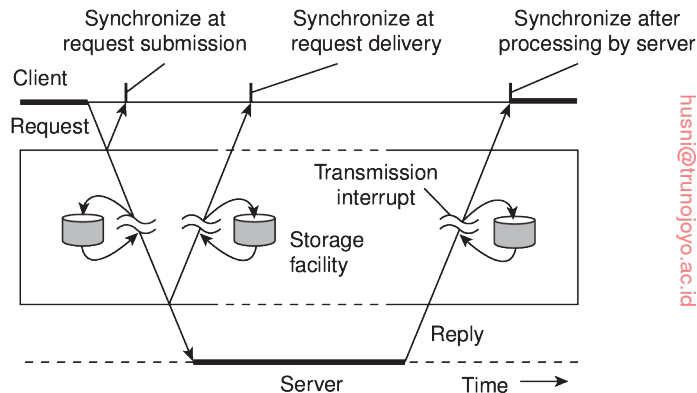


Figure 4.4: Viewing middleware as an intermediate (distributed) service in application-level communication.

An electronic mail system is a typical example in which communication is persistent. With **persistent communication**, a message that has been submitted for transmission is stored by the communication middleware as long as it takes to deliver it to the receiver. In this case, the middleware will store the message at one or several of the storage facilities shown in Figure 4.4. As a consequence, it is not necessary for the sending application to continue execution after submitting the message. Likewise, the receiving application need not be executing when the message is submitted.

In contrast, with **transient communication**, a message is stored by the communication system only as long as the sending and receiving application are executing. More precisely, in terms of Figure 4.4, if the middleware cannot

deliver a message due to a transmission interrupt, or because the recipient is currently not active, it will simply be discarded. Typically, all transport-level communication services offer only transient communication. In this case, the communication system consists of traditional store-and-forward routers. If a router cannot deliver a message to the next one or the destination host, it will simply drop the message.

Besides being persistent or transient, communication can also be asynchronous or synchronous. The characteristic feature of **asynchronous communication** is that a sender continues immediately after it has submitted its message for transmission. This means that the message is (temporarily) stored immediately by the middleware upon submission. With **synchronous communication**, the sender is blocked until its request is known to be accepted. There are essentially three points where synchronization can take place. First, the sender may be blocked until the middleware notifies that it will take over transmission of the request. Second, the sender may synchronize until its request has been delivered to the intended recipient. Third, synchronization may take place by letting the sender wait until its request has been fully processed, that is, up to the time that the recipient returns a response.

Various combinations of persistence and synchronization occur in practice. Popular ones are persistence in combination with synchronization at request submission, which is a common scheme for many message-queuing systems, which we discuss later in this chapter. Likewise, transient communication with synchronization after the request has been fully processed is also widely used. This scheme corresponds with remote procedure calls, which we discuss next.

4.2 Remote procedure call

Many distributed systems have been based on explicit message exchange between processes. However, the operations send and receive do not conceal communication at all, which is important to achieve access transparency in distributed systems. This problem has long been known, but little was done about it until researchers in the 1980s [Birrell and Nelson, 1984] introduced a completely different way of handling communication. Although the idea is refreshingly simple (once someone has thought of it), the implications are often subtle. In this section we will examine the concept, its implementation, its strengths, and its weaknesses.

In a nutshell, the proposal was to allow programs to call procedures located on other machines. When a process on machine A calls a procedure on machine B, the calling process on A is suspended, and execution of the called procedure takes place on B. Information can be transported from the caller to the callee in the parameters and can come back in the procedure result. No message passing at all is visible to the programmer. This method is known as

Remote Procedure Call, or often just RPC.

While the basic idea sounds simple and elegant, subtle problems exist. To start with, because the calling and called procedures run on different machines, they execute in different address spaces, which causes complications. Parameters and results also have to be passed, which can be complicated, especially if the machines are not identical. Finally, either or both machines can crash and each of the possible failures causes different problems. Still, most of these can be dealt with, and RPC is a widely-used technique that underlies many distributed systems.

Basic RPC operation

The idea behind RPC is to make a remote procedure call look as much as possible like a local one. In other words, we want RPC to be transparent—the calling procedure should not be aware that the called procedure is executing on a different machine or vice versa. Suppose that a program has access to a database that allows it to append data to a stored list, after which it returns a reference to the modified list. The operation is made available to a program by means of a routine `append`:

```
newlist = append(data, dbList)
```

In a traditional (single-processor) system, `append` is extracted from a library by the linker and inserted into the object program. In principle, it can be a short procedure, which could be implemented by a few file operations for accessing the database.

Even though `append` eventually does only a few basic file operations, it is called in the usual way, by pushing its parameters onto the stack. The programmer does not know the implementation details of `append`, and this is, of course, how it is supposed to be.

Note 4.2 (More information: Conventional procedure calls)

To understand how RPC works and some of its pitfalls, it may help to first understand how a conventional (i.e., single machine) procedure call works. Consider the following operation.

```
newlist = append(data, dbList);
```

We assume that the purpose of this call is to take a *globally* defined list object, referred here to as `dbList`, and append a simple data element to it represented by the variable `data`. An important observation is that in various programming languages such as C, `dbList` is implemented as a *reference* to a list object (i.e., a pointer), whereas `data` may be represented directly by its value (which we assume to be the case here). When calling `append`, both the representations of `data` and `dbList` are pushed onto the stack, making those representations accessible to the implementation of `append`. For `data`, this means the variable follows a

copy-by-value policy, the policy for `dblist` is **copy-by-reference**. What happens before and during the call is shown in Figure 4.5.

Several things are worth noting. For one, a value parameter such as `data`, is just an initialized local variable. The called procedure may modify it, but such changes do not affect the original value at the calling side.

When a parameter like `dblist` is actually a pointer to a variable rather than the value of the variable, something else happens. What is pushed onto the stack is the address of the list object as stored in main memory. When the value of `data` is appended to the list, a call to `append` *does* modify the list object. The difference between call-by-value and call-by-reference is quite important for RPC.

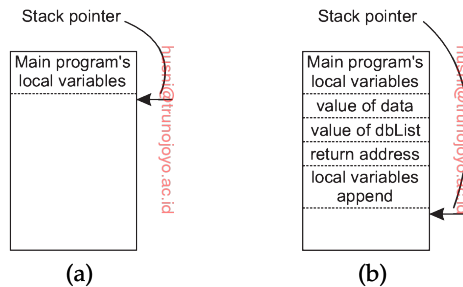


Figure 4.5: (a) Parameter passing in a local procedure call: the stack before the call to `append`. (b) The stack while the called procedure is active.

One other parameter passing mechanism also exists, although it is not used in most programming languages. It is called **call-by-copy/restore**. It consists of having the variable copied to the stack by the caller, as in call-by-value, and then copied back after the call, overwriting the caller's original value. Under most conditions, this achieves exactly the same effect as call-by-reference, but in some situations, such as the same parameter being present multiple times in the parameter list, the semantics are different.

The decision of which parameter passing mechanism to use is normally made by the language designers and is a fixed property of the language. Sometimes it depends on the data type being passed. In C, for example, integers and other scalar types are always passed by value, whereas arrays are always passed by reference. Some Ada compilers use copy/restore for inout parameters, but others use call-by-reference. The language definition permits either choice, which makes the semantics a bit fuzzy. In Python, all variables are passed by reference, but some actually get copied to local variables, thus mimicking the behavior of copy-by-value.

RPC achieves its transparency in an analogous way. When `append` is actually a remote procedure, a different version of `append`, called a **client stub**, is offered to the calling client. Like the original one, it, too, is called using a normal calling sequence. However, unlike the original one, it does not perform an `append` operation. Instead, it packs the parameters into a message

and requests that message to be sent to the server as illustrated in Figure 4.6. Following the call to send, the client stub calls receive, blocking itself until the reply comes back.

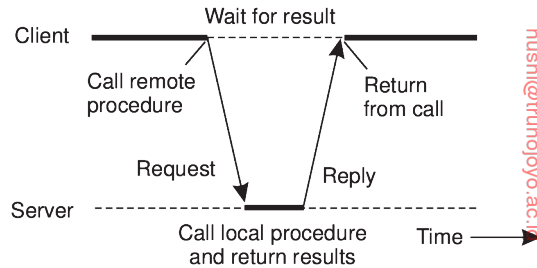


Figure 4.6: The principle of RPC between a client and server program.

When the message arrives at the server, the server's operating system passes it to a **server stub**. A server stub is the server-side equivalent of a client stub: it is a piece of code that transforms requests coming in over the network into local procedure calls. Typically the server stub will have called receive and be blocked waiting for incoming messages. The server stub unpacks the parameters from the message and then calls the server procedure in the usual way. From the server's point of view, it is as though it is being called directly by the client—the parameters and return address are all on the stack where they belong and nothing seems unusual. The server performs its work and then returns the result to the caller (in this case the server stub) in the usual way.

When the server stub gets control back after the call has completed, it packs the result in a message and calls send to return it to the client. After that, the server stub usually does a call to receive again, to wait for the next incoming request.

When the result message arrives at the client's machine, the operating system passes it through the receive operation, which had been called previously, to the client stub, and the client process is subsequently unblocked. The client stub inspects the message, unpacks the result, copies it to its caller, and returns in the usual way. When the caller gets control following the call to append, all it knows is that it appended some data to a list. It has no idea that the work was done remotely at another machine.

This blissful ignorance on the part of the client is the beauty of the whole scheme. As far as it is concerned, remote services are accessed by making ordinary (i.e., local) procedure calls, not by calling send and receive. All the details of the message passing are hidden away in the two library procedures, just as the details of actually making system calls are hidden away in traditional libraries.

To summarize, a remote procedure call occurs in the following steps:

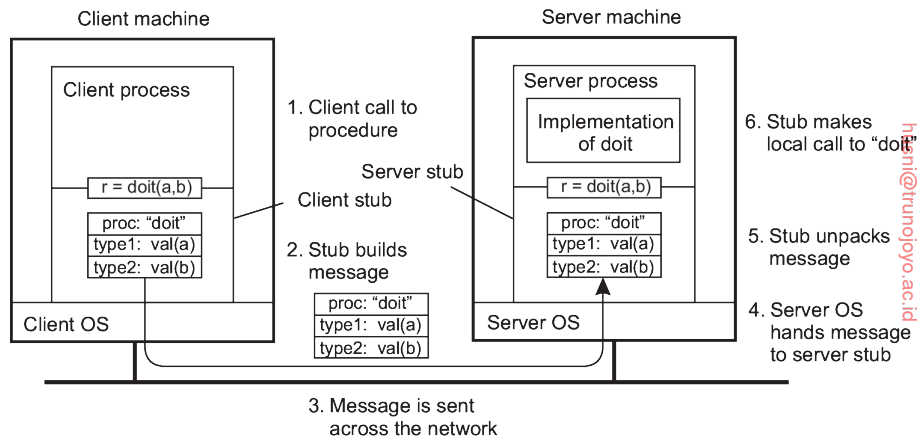


Figure 4.7: The steps involved in calling a remote procedure `doit(a,b)`. The return path for the result is not shown.

1. The client procedure calls the client stub in the normal way.
2. The client stub builds a message and calls the local operating system.
3. The client's OS sends the message to the remote OS.
4. The remote OS gives the message to the server stub.
5. The server stub unpacks the parameter(s) and calls the server.
6. The server does the work and returns the result to the stub.
7. The server stub packs the result in a message and calls its local OS.
8. The server's OS sends the message to the client's OS.
9. The client's OS gives the message to the client stub.
10. The stub unpacks the result and returns it to the client.

The first steps are shown in Figure 4.7 for an abstract two-parameter procedure `doit(a,b)`, where we assume that parameter `a` is of type `type1`, and `b` of type `type2`. The net effect of all these steps is to convert the local call by the client procedure to the client stub, to a local call to the server procedure without either client or server being aware of the intermediate steps or the existence of the network.

Note 4.3 (More information: An example in Python)

To make matters concrete, let us consider how a remote procedure call could be implemented for the operation `append` discussed previously. Take a look at the Python code shown in Figure 4.8 (from which we omit nonessential code fragments).

The class `DBList` is a simple representation of a list object, mimicking what one would expect to see in a version that would be found in a database environment. The client stub, represented by the class `Client`, consists of an implementation

of `append`. When called with parameters `data` and `dbList`, the following happens. The call is transformed into a tuple `(APPEND, data, dbList)` containing all the information the server would need to do its work. The client stub then sends the request off to the server, and subsequently waits for the response. When the response comes in, it finishes by passing the result to the program that initially called the stub.

```

1  import channel, pickle
2
3  class DBList:
4      def append(self, data):
5          self.value = self.value + [data]
6          return self
7
8  class Client:
9      def append(self, data, dbList):
10         msglst = (APPEND, data, dbList)      # message payload
11         self.chan.sendTo(self.server, msglst) # send msg to server
12         msgrcv = self.chan.recvFrom(self.server) # wait for response
13         return msgrcv[1]                    # pass it to caller
14
15  class Server:
16      def append(self, data, dbList):
17          return dbList.append(data)
18
19      def run(self):
20          while True:
21              msgreq = self.chan.recvFromAny() # wait for any request
22              client = msgreq[0]                # see who is the caller
23              msgrpc = msgreq[1]                # fetch call & parameters
24              if APPEND == msgrpc[0]:           # check what is being requested
25                  result = self.append(msgrpc[1], msgrpc[2]) # do local call
26                  self.chan.sendTo([client], result)          # return response

```

Figure 4.8: A simple RPC example for operation `append`.

On the server side, we see that in the server stub, the server waits for any incoming message, and inspects which operation it is required to call. Assuming it received a request to call `append`, it then simply does a *local* call to its implementation of `append` with the appropriate parameters as also found in the request tuple. The result is then sent off to the client.

Parameter passing

The function of the client stub is to take its parameters, pack them into a message, and send them to the server stub. While this sounds straightforward, it is not quite as simple as it at first appears.

Packing parameters into a message is called **parameter marshaling**. Returning to our `append` operation, we thus need to ensure that its two parame-

ters (data and dbList) are sent over the network and correctly interpreted by the server. The thing to realize here is that, in the end, the server will just be seeing a series of bytes coming in that constitute the original message sent by the client. However, no additional information on what those bytes mean is normally provided with the message, let alone that we would be facing the same problem again: how should the meta-information be recognized as such by the server?

Besides this interpretation problem, we also need to handle the case that the placement of bytes in memory may differ between machine architectures. In particular, we need to account for the fact that some machines, such as the Intel Pentium, number their bytes from right to left, whereas many others, such as the older ARM processors, number them the other way (ARM now supports both). The Intel format is called **little endian** and the (older) ARM format is called **big endian**. Byte ordering is also important for networking: also here we can witness that machines may use a different ordering when transmitting (and thus receiving) bits and bytes. However, big endian is what is normally used for transferring bytes across a network.

The solution to this problem is to transform data that is to be sent to a machine- and network-independent format, next to making sure that both communicating parties expect the same *message data type* to be transmitted. The latter can typically be solved at the level of programming languages. The former is accomplished by using machine-*dependent* routines that transform data to and from machine- and network-independent formats.

Marshaling and unmarshaling is all about this transformation to neutral formats and forms an essential part of remote procedure calls.

We now come to a difficult problem: How are pointers, or in general, references passed? The answer is: only with the greatest of difficulty, if at all. A pointer is meaningful only within the address space of the process in which it is being used. Getting back to our append example, we stated that the second parameter, dbList, is implemented by means of a *reference* to a list stored in a database. If that reference is just a pointer to a local data structure somewhere in the caller's main memory, we cannot simply pass it to the server. The transferred pointer value will most likely be referring to something completely different.

Note 4.4 (More information: An example in Python revisited)

It is not difficult to see that the solution to remote procedure calling as shown in Figure 4.8 will not work in general. Only if the client and server are operating on machines that obey the same byte-ordering rules *and* have the same machine representations for data structures, will the exchange of messages as shown lead to correct interpretations. A robust solution is shown in Figure 4.9 (where we again have omitted code for brevity).

In this example, we use the Python pickle library for marshaling and unmarshaling data structures. Note that the code hardly changes in comparison to what we have shown in Figure 4.8. The only changes occur just before sending, and after receiving a message. Also note that both client and server are programmed to work on the same data structures, as we discussed above.

```

1  import channel, pickle
2
3  class Client:
4      def append(self, data, dbList):
5          msglst = (APPEND, data, dbList)          # message payload
6          msgsnd = pickle.dumps(msglst)           # wrap call
7          self.chan.sendTo(self.server, msgsnd)    # send request to server
8          msgrcv = self.chan.recvFrom(self.server) # wait for response
9          retval = pickle.loads(msgrcv[1])         # unwrap return value
10         return retval                            # pass it to caller
11
12  class Server:
13      def run(self):
14          while True:
15              msgreq = self.chan.recvFromAny() # wait for any request
16              client = msgreq[0]               # see who is the caller
17              msgrpc = pickle.loads(msgreq[1]) # unwrap the call
18              if APPEND == msgrpc[0]:          # check what is being requested
19                  result = self.append(msgrpc[1], msgrpc[2]) # do local call
20                  msgres = pickle.dumps(result) # wrap the result
21                  self.chan.sendTo([client], msgres) # send response

```

Figure 4.9: A simple RPC example for operation append, but now with proper marshaling.

One solution is just to forbid pointers and reference parameters in general. However, these are so important that this solution is highly undesirable. In fact, it is often not necessary either. First, reference parameters are often used with fixed-sized data types, such as static arrays, or with dynamic data types for which it is easy to compute their size at runtime, such as strings or dynamic arrays. In such cases, we can simply copy the entire data structure to which the parameter is referring, effectively replacing the copy-by-reference mechanism by copy-by-value/restore. Although this is semantically not always identical, it frequently is good enough. An obvious optimization is that when the client stub knows the referred data will be only read, there is no need to copy it back when the call has finished. Copy-by-value is thus good enough.

More intricate data types can often be supported as well, and certainly if a programming language supports those data types. For example, a language such as Python or Java supports user-defined classes, allowing a language system to provide fully automated marshaling and unmarshaling of those data types. Note, however, that as soon as we are dealing with very large, nested,

or otherwise intricate dynamic data structures, automatic (un)marshaling may not be available, or even desirable.

The problem with pointers and references as discussed so far, is that they make only locally sense: they refer to memory locations that have meaning only to the calling process. Problems can be alleviated by using *global* references: references that are meaningful to the calling and the called process. For example, if the client and the server have access to the same file system, passing a file handle instead of a pointer may do the trick. There is one important observation: both processes need to know exactly what to do when a global reference is passed. In other words, if we consider a global reference having an associated data type, the calling and called process should have exactly the same picture of the operations that can be performed. Moreover, both processes should have agreement on exactly what to do when a file handle is passed. Again, these are typically issues that can be solved by proper programming-language support.

Note 4.5 (Advanced: Parameter passing in object-based systems)

Object-based systems often use global references. Consider the situation that all objects in the system can be accessed from remote machines. In that case, we can consistently use object references as parameters in method invocations. References are passed by value, and thus copied from one machine to the other. When a process is given an object reference as the result of a method invocation, it can simply bind to the object referred to when needed later (see also Section 2.1).

Unfortunately, using only distributed objects can be highly inefficient, especially when objects are small, such as integers, or worse yet, Booleans. Each invocation by a client that is not co-located in the same server as the object, generates a request between different address spaces or, even worse, between different machines. Therefore, references to remote objects and those to local objects are often treated differently.

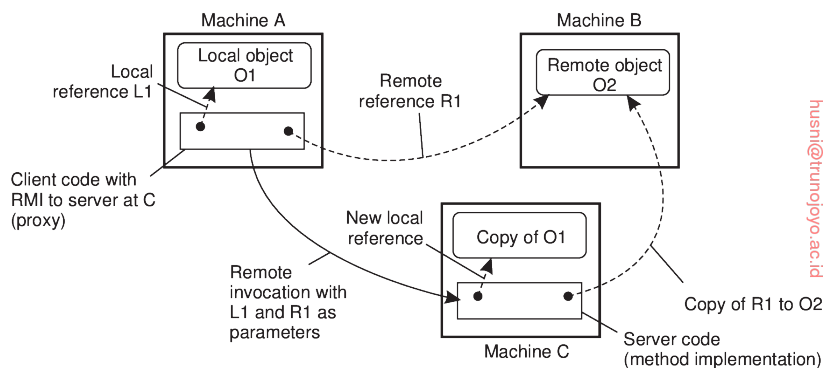


Figure 4.10: Passing an object by reference or by value.

When invoking a method with an object reference as parameter, that reference

is copied and passed as a value parameter only when it refers to a remote object. In this case, the object is literally passed by reference. However, when the reference refers to a local object, that is an object in the same address space as the client, the referred object is copied as a whole and passed along with the invocation. In other words, the object is passed by value.

These two situations are illustrated in Figure 4.10 which shows a client program running on machine A, and a server program on machine C. The client has a reference to a local object O1 that it uses as a parameter when calling the server program on machine C. In addition, it holds a reference to a remote object O2 residing at machine B, which is also used as a parameter. When calling the server, a copy of O1 is passed to the server on machine C, along with only a copy of the reference to O2.

Note that whether we are dealing with a reference to a local object or a reference to a remote object can be highly transparent, such as in Java. In Java, the distinction is visible only because local objects are essentially of a different data type than remote objects. Otherwise, both types of references are treated very much the same (see also [Wollrath et al., 1996]). On the other hand, when using conventional programming languages such as C, a reference to a local object can be as simple as a pointer, which can never be used to refer to a remote object.

The side effect of invoking a method with an object reference as parameter is that we may be *copying* an object. Obviously, hiding this aspect is unacceptable, so that we are consequently forced to make an explicit distinction between local and distributed objects. Clearly, this distinction not only violates distribution transparency, but also makes it harder to write distributed applications.

We can now also easily explain how global references can be implemented when using portable, interpreted languages such as Python or Java: use the entire client stub as a reference. The key observation is that a client stub is often just another data structure that is compiled into (portable) bytecode. That compiled code can actually be transferred across the network and executed at the receiver's side. In other words, there is no need for explicit binding anymore; simply copying the client stub to the recipient is enough to allow the latter to invoke the associated server-side object.

RPC-based application support

From what we have explained so far, it is clear that hiding a remote procedure call requires that the caller and the callee agree on the format of the messages they exchange and that they follow the same steps when it comes to, for example, passing complex data structures. In other words, both sides in an RPC should follow the same protocol or the RPC will not work correctly. There are at least two ways in which RPC-based application development can be supported. The first one is to let a developer specify exactly what needs to be called remotely, from which complete client-side and server-side stubs can be generated. A second approach is to embed remote procedure calling as part of a programming-language environment.

Stub generation

Consider the function `someFunction` of Figure 4.11(a). It has three parameters, a character, a floating-point number, and an array of five integers. Assuming a word is four bytes, the RPC protocol might prescribe that we should transmit a character in the rightmost byte of a word (leaving the next three bytes empty), a float as a whole word, and an array as a group of words equal to the array length, preceded by a word giving the length, as shown in Figure 4.11(b). Thus given these rules, the client stub for `someFunction` knows that it must use the format of Figure 4.11(b), and the server stub knows that incoming messages for `someFunction` will have the format of Figure 4.11(b).

```
void someFunction(char x; float y; int z[5])
```

(a)

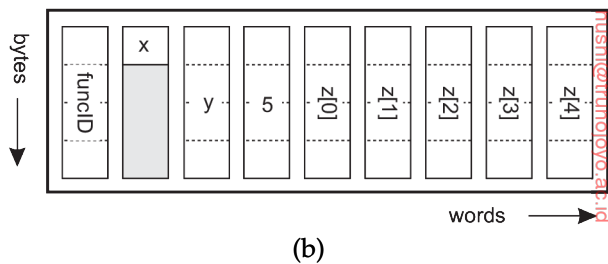


Figure 4.11: (a) A function. (b) The corresponding message, and the order in which bytes and words are sent across the network.

Defining the message format is one aspect of an RPC protocol, but it is not sufficient. What we also need is the client and the server to agree on the representation of simple data structures, such as integers, characters, Booleans, etc. For example, the protocol could prescribe that integers are represented in two's complement, characters in 16-bit Unicode, and floats in the IEEE standard #754 format, with everything stored in little endian. With this additional information, messages can be unambiguously interpreted.

With the encoding rules now pinned down to the last bit, the only thing that remains to be done is that the caller and callee agree on the actual exchange of messages. For example, it may be decided to use a connection-oriented transport service such as TCP/IP. An alternative is to use an unreliable datagram service and let the client and server implement an error control scheme as part of the RPC protocol. In practice, several variants exist, and it is up to the developer to indicate the preferred underlying communication service.

Once the RPC protocol has been fully defined, the client and server stubs

need to be implemented. Fortunately, stubs for the same protocol but different procedures normally differ only in their interface to the applications. An interface consists of a collection of procedures that can be called by a client, and which are implemented by a server. An interface is usually available in the same programming language as the one in which the client or server is written (although this is strictly speaking, not necessary). To simplify matters, interfaces are often specified by means of an **Interface Definition Language (IDL)**. An interface specified in such an IDL is then subsequently compiled into a client stub and a server stub, along with the appropriate compile-time or run-time interfaces.

Practice shows that using an interface definition language considerably simplifies client-server applications based on RPCs. Because it is easy to fully generate client and server stubs, all RPC-based middleware systems offer an IDL to support application development. In some cases, using the IDL is even mandatory.

Language-based support

The approach described up until now is largely independent of a specific programming language. As an alternative, we can also embed remote procedure calling into a language itself. The main benefit is that application development often becomes much simpler. Also, reaching a high degree of access transparency is often simpler as many issues related to parameter passing can be circumvented altogether.

A well-known example in which remote procedure calling is fully embedded is Java, where an RPC is referred to as a **remote method invocation (RMI)**. In essence, a client being executed by its own (Java) virtual machine can invoke a method of an object managed by another virtual machine. By simply reading an application's source code, it may be hard or even impossible to see whether a method invocation is to a local or to a remote object.

Note 4.6 (More information: Language-based RPC in Python)

Let us see by an example of how remote procedure calling can be integrated in a language. We have been using the Python language for most of our examples, and will continue to do so now as well. In Figure 4.12(a) we show a simple server for our DBList data structure. In this case, it has two *exposed* operations: `exposed_append` for appending elements, and `exposed_value` to display what is currently in the list. We use the Python RPyC package for embedding RPCs.

The client is shown in Figure 4.12(b). When a connection is made to the server, a new instance of DBList will be created and the client can immediately append values to the list. The exposed operations can be called without further ado.

```

1 import rpyc
2 from rpyc.utils.server import ForkingServer
3
4 class DBList(rpyc.Service):
5     value = []
6
7     def exposed_append(self, data):
8         self.value = self.value + [data]
9         return self.value
10
11     def exposed_value(self):
12         return self.value
13
14 if __name__ == "__main__":
15     server = ForkingServer(DBList, port = 12345)
16     server.start()

```

Figure 4.12: (a) Embedding RPCs in a language: a server.

```

1 import rpyc
2
3 class Client:
4     conn = rpyc.connect(SERVER, PORT) # Connect to the server
5     conn.root.exposed_append(2)        # Call an exposed operation,
6     conn.root.exposed_append(4)        # and append two elements
7     print conn.root.exposed_value()    # Print the result

```

Figure 4.12: (b) Embedding RPCs in a language: a client.

This example also illustrates a subtle issue: apparently the instance of `DBList` created for the client is new, and unique. In other words, as soon as the client breaks the connection to the server, the list will be lost. It is a **transient object** so to say, and special measures will need to be taken to make it a **persistent object**.

Variations on RPC

As in conventional procedure calls, when a client calls a remote procedure, the client will block until a reply is returned. This strict request-reply behavior is unnecessary when there is no result to return, or may hinder efficiency when multiple RPCs need to be performed. In the following we look at two variations on the RPC scheme we have discussed so far.

Asynchronous RPC

To support situations in which there is simply no result to return to the client, RPC systems may provide facilities for what are called **asynchronous RPCs**. With asynchronous RPCs, the server, in principle, immediately sends a reply

back to the client the moment the RPC request is received, after which it locally calls the requested procedure. The reply acts as an acknowledgment to the client that the server is going to process the RPC. The client will continue without further blocking as soon as it has received the server's acknowledgment. Figure 4.13(b) shows how client and server interact in the case of asynchronous RPCs. For comparison, Figure 4.13(a) shows the normal request-reply behavior.

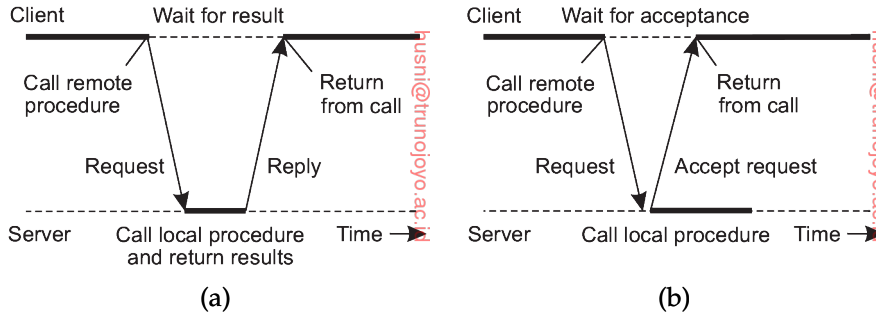


Figure 4.13: (a) The interaction between client and server in a traditional RPC. (b) The interaction using asynchronous RPC.

Asynchronous RPCs can also be useful when a reply will be returned but the client is not prepared to wait for it and do nothing in the meantime. A typical case is when a client needs to contact several servers independently. In that case, it can send the call requests one after the other, effectively establishing that the servers operate more or less in parallel. After all call requests have been sent, the client can start waiting for the various results to be returned. In cases such as these, it makes sense to organize the communication between the client and server through an asynchronous RPC combined with a **callback**, as shown in Figure 4.14. In this scheme, also referred to as **deferred synchronous RPC**, the client first calls the server, waits for the acceptance, and continues. When the results become available, the server sends a response message that leads to a callback at the client's side. A callback is a user-defined function that is invoked when a special event happens, such as an incoming message. A straightforward implementation is to spawn a separate thread and let it block on the occurrence of the event while the main process continues. When the event occurs, the thread is unblocked and calls the function.

It should be noted that variants of asynchronous RPCs exist in which the client continues executing immediately after sending the request to the server. In other words, the client does not wait for an acknowledgment of the server's acceptance of the request. We refer to such RPCs as **one-way RPCs**. The problem with this approach is that when reliability is not guaranteed, the client cannot know for sure whether or not its request will be processed.

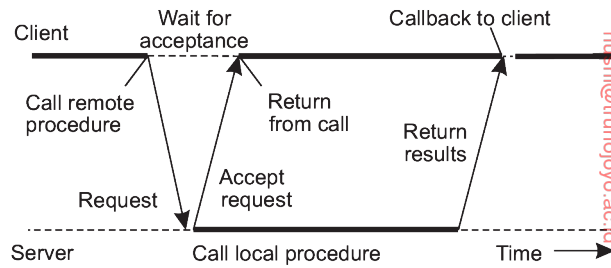


Figure 4.14: A client and server interacting through asynchronous RPCs.

We return to these matters in Chapter 8. Likewise, in the case of deferred synchronous RPC, the client may poll the server to see whether the results are available yet, instead of letting the server calling back the client.

Multicast RPC

Asynchronous and deferred synchronous RPCs facilitate another alternative to remote procedure calls, namely executing multiple RPCs at the same time. Adopting the one-way RPCs (i.e., when a server does not tell the client it has accepted its call request but immediately starts processing it), a **multicast RPC** boils down to sending an RPC request to a group of servers. This principle is shown in Figure 4.15. In this example, the client sends a request to two servers, who subsequently process that request independently and in parallel. When done, the result is returned to the client where a callback takes place.

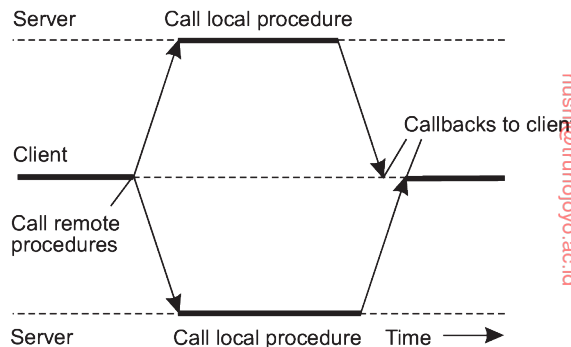


Figure 4.15: The principle of a multicast RPC.

There are several issues that we need to consider. First, as before, the client application may be unaware of the fact that an RPC is actually being forwarded to more than one server. For example, to increase fault tolerance, we may decide to have all operations executed by a backup server who can take over when the main server fails. That a server has been replicated can be

completely hidden from a client application by an appropriate stub. Yet even the stub need not be aware that the server is replicated, for example, because we are using a transport-level multicast address.

Second, we need to consider what to do with the responses. In particular, will the client proceed after all responses have been received, or wait just for one? It all depends. When the server has been replicated for fault tolerance, we may decide to wait for just the first response, or perhaps until a majority of the servers returns the same result. On the other hand, if the servers have been replicated to do the same work but on different parts of the input, their results may need to be merged before the client can continue. Again, such matters can be hidden in the client-side stub, yet the application developer will, at the very least, have to specify the purpose of the multicast RPC.

Example: DCE RPC

Remote procedure calls have been widely adopted as the basis of middleware and distributed systems in general. In this section, we take a closer look at the **Distributed Computing Environment (DCE)** which was developed by the Open Software Foundation (OSF), now called The Open Group. It forms the basis for Microsoft's distributed computing environment DCOM [Eddon and Eddon, 1998] and used in Samba, a file server and accompanying protocol suite allowing the Windows file system to be accessed through remote procedure calls from non-Windows systems.

Although DCE RPC is arguably not the most modern way of managing RPCs, it is worthwhile discussing some of its details, notably because it is representative for most traditional RPC systems that use a combination of interface specifications and explicit bindings to various programming languages. We start with a brief introduction to DCE, after which we consider its principal workings. Details on how to develop RPC-based applications can be found in [Stevens, 1999].

Introduction to DCE

DCE is a true middleware system in that it is designed to execute as a layer of abstraction between existing (network) operating systems and distributed applications. Initially designed for Unix, it has now been ported to all major operating systems. The idea is that the customer can take a collection of existing machines, add the DCE software, and then be able to run distributed applications, all without disturbing existing (nondistributed) applications. Although most of the DCE package runs in user space, in some configurations a piece (part of the distributed file system) must be added to the kernel of the underlying operating system.

The programming model underlying DCE is the client-server model. User processes act as clients to access remote services provided by server processes.

Some of these services are part of DCE itself, but others belong to the applications and are written by the application programmers. All communication between clients and servers takes place by means of RPCs.

Goals of DCE RPC

The goals of the DCE RPC system are relatively traditional. First and foremost, the RPC system makes it possible for a client to access a remote service by simply calling a local procedure. This interface makes it possible for client (i.e., application) programs to be written in a simple way, familiar to most programmers. It also makes it easy to have large volumes of existing code run in a distributed environment with few, if any, changes.

It is up to the RPC system to hide all the details from the clients, and, to some extent, from the servers as well. To start with, the RPC system can automatically locate the correct server, and subsequently set up the communication between client and server software (generally called **binding**). It can also handle the message transport in both directions, fragmenting and reassembling them as needed (e.g., if one of the parameters is a large array). Finally, the RPC system can automatically handle data type conversions between the client and the server, even if they run on different architectures and have a different byte ordering.

As a consequence of the RPC system's ability to hide the details, clients and servers are highly independent of one another. A client can be written in Java and a server in C, or vice versa. A client and server can run on different hardware platforms and use different operating systems. A variety of network protocols and data representations are also supported, all without any intervention from the client or server.

Writing a Client and a Server

The DCE RPC system consists of a number of components, including languages, libraries, daemons, and utility programs, among others. Together these make it possible to write clients and servers. In this section we will describe the pieces and how they fit together. The entire process of writing and using an RPC client and server is summarized in Figure 4.16.

In a client-server system, the glue that holds everything together is the interface definition, as specified in the **Interface Definition Language**, or **IDL**. It permits procedure declarations in a form closely resembling function prototypes in ANSI C. IDL files can also contain type definitions, constant declarations, and other information needed to correctly marshal parameters and unmarshal results. Ideally, the interface definition should also contain a formal definition of what the procedures do, but such a definition is beyond the current state of the art, so the interface definition just defines the syntax

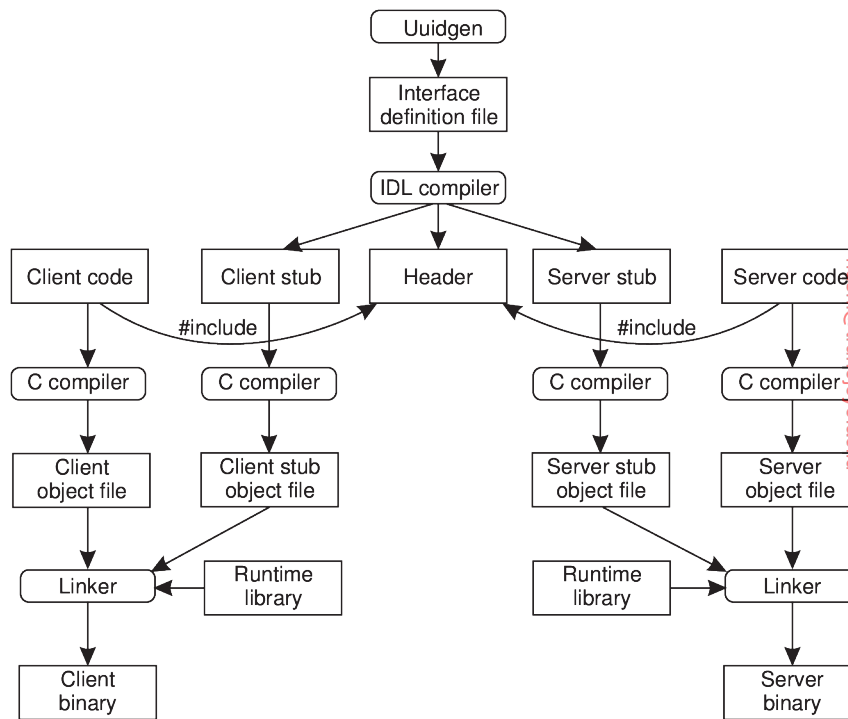


Figure 4.16: The steps in writing a client and a server in DCE RPC.

of the calls, not their semantics. At best the writer can add a few comments describing what the procedures do.

A crucial element in every IDL file is a globally unique identifier for the specified interface. The client sends this identifier in the first RPC message and the server verifies that it is correct. In this way, if a client inadvertently tries to bind to the wrong server, or even to an older version of the right server, the server will detect the error and the binding will not take place.

Interface definitions and unique identifiers are closely related in DCE. As illustrated in Figure 4.16, the first step in writing a client/server application is usually calling the `uuidgen` program, asking it to generate a prototype IDL file containing an interface identifier guaranteed never to be used again in any interface generated anywhere by `uuidgen`. Uniqueness is ensured by encoding in it the location and time of creation. It consists of a 128-bit binary number represented in the IDL file as an ASCII string in hexadecimal.

The next step is editing the IDL file, filling in the names of the remote procedures and their parameters. It is worth noting that RPC is not totally transparent. For example, the client and server cannot share global variables. The IDL rules make it impossible to express constructs that are not supported.

When the IDL file is complete, the IDL compiler is called to process it. The output of the IDL compiler consists of three files:

- A header file (e.g., `interface.h`, in C terms).
- The client stub.
- The server stub.

The header file contains the unique identifier, type definitions, constant definitions, and function prototypes. It should be included (using `#include`) in both the client and server code. The client stub contains the actual procedures that the client program will call. These procedures are the ones responsible for collecting and packing the parameters into the outgoing message and then calling the runtime system to send it. The client stub also handles unpacking the reply and returning values to the client. The server stub contains the procedures called by the runtime system on the server machine when an incoming message arrives. These, in turn, call the actual server procedures that do the work.

The next step is for the application writer to write the client and server code. Both of these are then compiled, as are the two stub procedures. The resulting client code and client stub object files are then linked with the runtime library to produce the executable binary for the client. Similarly, the server code and server stub are compiled and linked to produce the server's binary. At runtime, the client and server are started so that the application is actually executed as well.

Binding a client to a server

To allow a client to call a server, it is necessary that the server has been registered and is prepared to accept incoming calls. Registration of a server makes it possible for a client to locate the server and bind to it. Finding the location of the server is done in two steps:

1. Locate the server's machine.
2. Locate the server (i.e., the correct process) on that machine.

The second step is somewhat subtle. Basically, what it comes down to is that to communicate with a server, the client needs to know a **port** on the server's machine to which it can send messages. A port is used by the server's operating system to distinguish incoming messages for different processes. In DCE, a table of (server,port) pairs is maintained on each server machine by a process called the **DCE daemon**. Before it becomes available for incoming requests, the server must ask the operating system for a port. It then registers this port with the DCE daemon. The DCE daemon records this information (including which protocols the server speaks) in the port table for future use.

The server also registers with the directory service by providing it the network address of the server's machine and a name under which the server can be looked up. Binding a client to a server proceeds as shown in Figure 4.17.

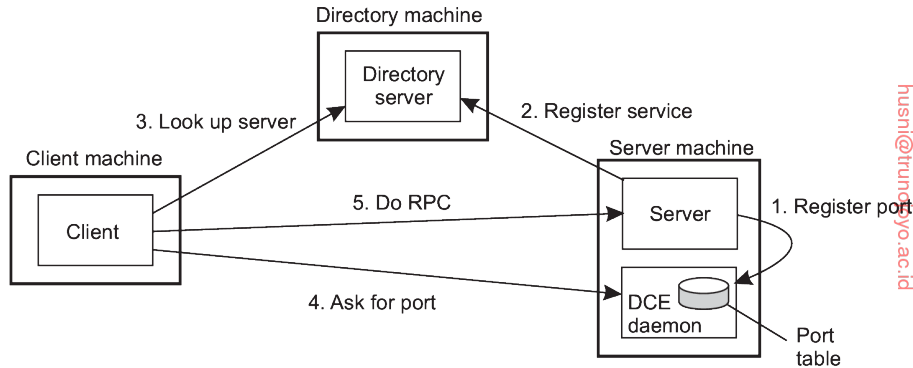


Figure 4.17: Client-to-server binding in DCE.

Let us assume that the client wants to bind to a video server that is locally known under the name `/local/multimedia/video/movies`. It passes this name to the directory server, which returns the network address of the machine running the video server. The client then goes to the DCE daemon on that machine (which has a well-known port), and asks it to look up the port of the video server in its port table. Armed with this information, the RPC can now take place. On subsequent RPCs this lookup is not needed. DCE also gives clients the ability to do more sophisticated searches for a suitable server when that is needed. Secure RPC is also an option where confidentiality or data integrity is crucial.

Performing an RPC

The actual RPC is carried out transparently and in the usual way. The client stub marshals the parameters to the runtime library for transmission using the protocol chosen at binding time. When a message arrives at the server side, it is routed to the correct server based on the port contained in the incoming message. The runtime library passes the message to the server stub, which unmarshals the parameters and calls the server. The reply goes back by the reverse route.

DCE provides several semantic options. The default is **at-most-once operation**, in which case no call is ever carried out more than once, even in the presence of system crashes. In practice, what this means is that if a server crashes during an RPC and then recovers quickly, the client does not repeat the operation, for fear that it might already have been carried out once.

Alternatively, it is possible to mark a remote procedure as **idempotent** (in the IDL file), in which case it can be repeated multiple times without harm. For example, reading a specified block from a file can be tried over and over until it succeeds. When an idempotent RPC fails due to a server crash, the client can wait until the server reboots and then try again. Other semantics are also available (but rarely used), including broadcasting the RPC to all the machines on the local network. We return to RPC semantics in Section 8.3 when discussing RPC in the presence of failures.

4.3 Message-oriented communication

Remote procedure calls and remote object invocations contribute to hiding communication in distributed systems, that is, they enhance access transparency. Unfortunately, neither mechanism is always appropriate. In particular, when it cannot be assumed that the receiving side is executing at the time a request is issued, alternative communication services are needed. Likewise, the inherent synchronous nature of RPCs, by which a client is blocked until its request has been processed, may need to be replaced by something else.

That something else is messaging. In this section we concentrate on message-oriented communication in distributed systems by first taking a closer look at what exactly synchronous behavior is and what its implications are. Then, we discuss messaging systems that assume that parties are executing at the time of communication. Finally, we will examine message-queuing systems that allow processes to exchange information, even if the other party is not executing at the time communication is initiated.

Simple transient messaging with sockets

Many distributed systems and applications are built directly on top of the simple message-oriented model offered by the transport layer. To better understand and appreciate the message-oriented systems as part of middleware solutions, we first discuss messaging through transport-level sockets.

Special attention has been paid to standardizing the interface of the transport layer to allow programmers to make use of its entire suite of (messaging) protocols through a simple set of operations. Also, standard interfaces make it easier to port an application to a different machine. As an example, we briefly discuss the **socket interface** as introduced in the 1970s in Berkeley Unix, and which has been adopted as a POSIX standard (with only very few adaptations).

Conceptually, a **socket** is a communication end point to which an application can write data that are to be sent out over the underlying network, and from which incoming data can be read. A socket forms an abstraction over the actual port that is used by the local operating system for a specific transport

protocol. In the following text, we concentrate on the socket operations for TCP, which are shown in Figure 4.18.

Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection

Figure 4.18: The socket operations for TCP/IP.

Servers generally execute the first four operations, normally in the order given. When calling the socket operation, the caller creates a new communication end point for a specific transport protocol. Internally, creating a communication end point means that the local operating system reserves resources for sending and receiving messages for the specified protocol.

The bind operation associates a local address with the newly created socket. For example, a server should bind the IP address of its machine together with a (possibly well-known) port number to a socket. Binding tells the operating system that the server wants to receive messages only on the specified address and port. In the case of connection-oriented communication, the address is used to receive incoming connection requests.

The listen operation is called only in the case of connection-oriented communication. It is a nonblocking call that allows the local operating system to reserve enough buffers for a specified maximum number of pending connection requests that the caller is willing to accept.

A call to accept blocks the caller until a connection request arrives. When a request arrives, the local operating system creates a new socket with the same properties as the original one, and returns it to the caller. This approach will allow the server to, for example, fork off a process that will subsequently handle the actual communication through the new connection. The server can go back and wait for another connection request on the original socket.

Let us now take a look at the client side. Here, too, a socket must first be created using the socket operation, but explicitly binding the socket to a local address is not necessary, since the operating system can dynamically allocate a port when the connection is set up. The connect operation requires that the caller specifies the transport-level address to which a connection request is to be sent. The client is blocked until a connection has been set up successfully, after which both sides can start exchanging information through the send and

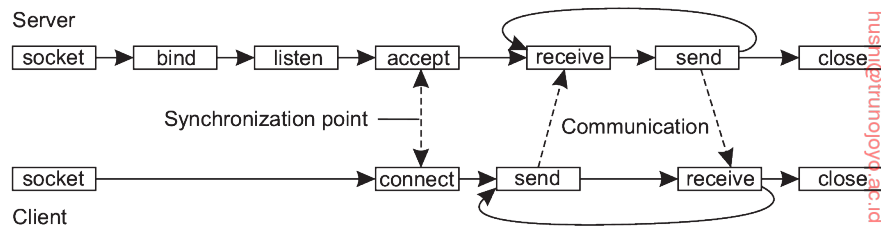


Figure 4.19: Connection-oriented communication pattern using sockets.

receive operations. Finally, closing a connection is symmetric when using sockets, and is established by having both the client and server call the `close` operation. Although there are many exceptions to the rule, the general pattern followed by a client and server for connection-oriented communication using sockets is as shown in Figure 4.19. Details on network programming using sockets and other interfaces in Unix can be found in [Stevens, 1998].

Note 4.7 (Example: A simple socket-based client-server system)

As an illustration of the recurring pattern in Figure 4.19, consider the simple socket-based client-server system shown below (see also Note 2.1). We see the server [Figure 4.20(a)] starting by creating a socket, and subsequently binding an address to that socket. It calls the `listen` operation, and waits for an incoming connection request. When the server accepts a connection, the socket library creates a separate connection, `conn`, which is used to receive data and send a response to the connected client. The server enters a loop receiving and sending messages, until no more data has been received. It then closes the connection.

```

1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.bind((HOST, PORT))
4 s.listen(1)
5 (conn, addr) = s.accept() # returns new socket and addr. client
6 while True:               # forever
7     data = conn.recv(1024) # receive data from client
8     if not data: break     # stop if client stopped
9     conn.send(str(data)+"*") # return sent data plus an "*"
10 conn.close()              # close the connection

```

Figure 4.20: (a) A simple socket-based client-server system: the server.

The client, shown in Figure 4.20(b), again follows the pattern from Figure 4.19. It creates a socket, and calls `connect` to request a connection with the server. Once the connection has been established, it sends a single message, waits for the response, and after printing the result, closes the connection.

```

1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.connect((HOST, PORT)) # connect to server (block until accepted)
4 s.send('Hello, world') # send same data
5 data = s.recv(1024) # receive the response
6 print data # print the result
7 s.close() # close the connection

```

Figure 4.20: (b) A simple socket-based client-server system: the client.

Note 4.8 (Advanced: Implementing stubs as global references revisited)

To provide a deeper insight in the working of sockets, let us look at a more elaborate example, namely the use of stubs as global references. We return to our example of implementing a shared list, which we now do by means of a list server, implemented in the form of the Python class shown in Figure 4.21(b). Figure 4.21(a) shows the stub implementation of a shared list. Again, we have omitted code for readability.

```

1 class DBClient:
2     def sendrecv(self, message):
3         sock = socket() # create a socket
4         sock.connect((self.host, self.port)) # connect to server
5         sock.send(pickle.dumps(message)) # send some data
6         result = pickle.loads(sock.recv(1024)) # receive the response
7         sock.close() # close the connection
8         return result
9
10    def create(self):
11        self.listID = self.sendrecv([CREATE])
12        return self.listID
13
14    def getValue(self):
15        return self.sendrecv([GETVALUE, self.listID])
16
17    def appendData(self, data):
18        return self.sendrecv([APPEND, data, self.listID])

```

Figure 4.21: (a) Implementing a list server in Python: the list structure.

The DBClient class represents a client-side stub that, once marshaled, can be passed between processes. It provides three operations associated with a list: create, getValue, and append, with obvious semantics. A DBClient is assumed to be associated with one specific list as managed by the server. An identifier for that list is returned when the list is created. Note how the (internal) sendrecv operation follows the client-side pattern explained in Figure 4.19.

The server maintains lists, as shown in Figure 4.21(b). Its internal data structure is a setOfLists with each element being a previously created list. The

server simply waits for incoming requests, unmarshals the request, and checks which operation is being requested. Results are sent back to the requesting client (which always issues the `sendrecv` operation implemented as part of `DBClient`). Again, we see that the server follows the pattern shown in Figure 4.19: it creates a socket, binds an address to it, informs the operating system to how many connections it should listen, and then waits to accept an incoming connection request. Once a connection has been established, the server receives data, sends a response, and closes the connection again.

```

1 class Server:
2     def __init__(self, port=PORT):
3         self.host = 'localhost'           # this machine
4         self.port = port                  # the port it will listen to
5         self.sock = socket()              # socket for incoming calls
6         self.sock.bind((self.host,self.port)) # bind socket to an address
7         self.sock.listen(5)               # max num of connections
8         self.setOfLists = {}              # init: no lists to manage
9
10    def run(self):
11        while True:
12            (conn, addr) = self.sock.accept() # accept incoming call
13            data = conn.recv(1024)           # fetch data from client
14            request = pickle.loads(data)      # unwrap the request
15            if request[0] == CREATE:          # create a list
16                listID = len(self.setOfLists) + 1 # allocate listID
17                self.setOfLists[listID] = []    # initialize to empty
18                conn.send(pickle.dumps(listID)) # return ID
19
20            elif request[0] == APPEND:        # append request
21                listID = request[2]           # fetch listID
22                data = request[1]            # fetch data to append
23                self.setOfLists[listID].append(data) # append it to the list
24                conn.send(pickle.dumps(OK))   # return an OK
25
26            elif request[0] == GETVALUE:     # read request
27                listID = request[1]          # fetch listID
28                result = self.setOfLists[listID] # get the elements
29                conn.send(pickle.dumps(result)) # return the list
30            conn.close()                     # close the connection

```

Figure 4.21: (b) Implementing a list server in Python: the server.

To use a stub as a global reference, we represent each client application by means of the class `Client` shown in Figure 4.21(c). The class is instantiated in the same process running the application (exemplified by the value of `self.host`), and will be listening on a specific port for messages from other applications, as well as the server. Otherwise, it merely sends and receives messages, coded through the operations `sendTo` and `recvAny`, respectively.

Now consider the code shown in Figure 4.21(d), which mimics two client applications. The first one creates a new list and appends data to it. Then note how `dbClient1` is simply sent to the other client. Under the hood, we now know

that it is marshaled in the operation `sendTo` (line 12) of class `Client` shown in Figure 4.21(c).

```

1 class Client:
2     def __init__(self, port):
3         self.host = 'localhost'           # this machine
4         self.port = port                 # port it will listen to
5         self.sock = socket()             # socket for incoming calls
6         self.sock.bind((self.host, self.port)) # bind socket to an address
7         self.sock.listen(2)              # max num connections
8
9     def sendTo(self, host, port, data):
10        sock = socket()
11        sock.connect((host, port))        # connect to server (blocking call)
12        sock.send(pickle.dumps(data))     # send some data
13        sock.close()
14
15    def recvAny(self):
16        (conn, addr) = self.sock.accept()
17        return conn.recv(1024)

```

Figure 4.21: (c) Implementing a list server in Python: the client.

```

1 pid = os.fork()
2 if pid == 0:
3     client1 = Client(CLIENT1)           # create client
4     dbClient1 = DBClient(HOST,PORT)     # create reference
5     dbClient1.create()                  # create new list
6     dbClient1.appendData('Client 1')    # append some data
7     client1.sendTo(HOSTCL2,CLIENT2,dbClient1) # send to other client
8
9 pid = os.fork()
10 if pid == 0:
11     client2 = Client(CLIENT2)           # create a new client
12     data = client2.recvAny()             # block until data is sent
13     dbClient2 = pickle.loads(data)       # receive reference
14     dbClient2.appendData('Client 2')     # append data to same list

```

Figure 4.21: (d) Passing stubs as references.

The second client simply waits for an incoming message (line 12), unmarshals the result, knowing that it is a `DBClient` instance, and subsequently appends some more data to the same list as the one the first client appended data. Indeed, an instance of `DBClient` is seen to be passed as global reference, seemingly along with all the operations that go with the associated class.

Advanced transient messaging

The standard socket-based approach toward transient messaging is very basic and as such, rather brittle: a mistake is easily made. Furthermore, sockets

essentially support only TCP or UDP, meaning that any extra facility for messaging needs to be implemented separately by an application programmer. In practice, we do often need more advanced approaches for message-oriented communication to make network programming easier, to expand beyond the functionality offered by existing networking protocols, to make better use of local resources, and so on.

Using messaging patterns: ZeroMQ

One approach toward making network programming easier is based on the observation that many messaging applications, or their components, can be effectively organized according to a few simple communication patterns. By subsequently providing enhancements to sockets for each of these patterns, it may become easier to develop a networked, distributed application. This approach has been followed in ZeroMQ and documented in [Hintjens, 2013; Akgul, 2013].

Like in the Berkeley approach, ZeroMQ also provides sockets through which all communication takes place. Actual message transmission generally takes place over TCP connections, and like TCP, all communication is essentially connection-oriented, meaning that a connection will first be set up between a sender and receiver before message transmission can take place. However, setting up, and maintaining connections is kept mostly under the hood: an application programmer need not bother with those issues. To further simplify matters, a socket may be bound to multiple addresses, effectively allowing a server to handle messages from very different sources through a single interface. For example, a server can listen to multiple ports using a single blocking receive operation. ZeroMQ sockets can thus support *many-to-one* communication instead of just *one-to-one* communication as is the case with standard Berkeley sockets. To complete the story: ZeroMQ sockets also support *one-to-many* communication, i.e., multicasting.

Essential to ZeroMQ is that communication is asynchronous: a sender will normally continue after having submitted a message to the underlying communication subsystem. An interesting side effect of combining asynchronous with connection-oriented communication, is that a process can request a connection setup, and subsequently send a message even if the recipient is not yet up-and-running and ready to accept incoming connection requests, let alone incoming messages. What happens, of course, is that a connection request and subsequent messages are queued at the sender's side, while a separate thread as part of ZeroMQ's library will take care that eventually the connection is set up and messages are transmitted to the recipient.

Simplifying matters, ZeroMQ establishes a higher level of abstraction in socket-based communication by *pairing* sockets: a specific type of socket used for sending messages is paired with a corresponding socket type for receiving messages. Each pair of socket types corresponds to a communication pattern.

The three most important communication patterns supported by ZeroMQ are *request-reply*, *publish-subscribe*, and *pipeline*.

The **request-reply pattern** is used in traditional client-server communication, like the ones normally used for remote procedure calls. A client application uses a **request socket** (of type REQ) to send a request message to a server and expects the latter to respond with an appropriate response. The server is assumed to use a **reply socket** (of type REP). The request-reply pattern simplifies matters for developers by avoiding the need to call the listen operation, as well as the accept operation. Moreover, when a server receives a message, a subsequent call to send is automatically targeted toward the original sender. Likewise, when a client calls the recv operation (for receiving a message) after having sent a message, ZeroMQ assumes the client is waiting for a response from the original recipient. Note that this approach was effectively encoded in the local sendrecv operation of Figure 4.21 discussed in Note 4.8.

Note 4.9 (Example: The request-reply pattern)

Let us look at a simple programming example to illustrate the request-reply pattern. Figure 4.22(a) shows a server that appends an asterisk to a received message. As before, it creates a socket, and binds it to a combination of a protocol (in this case TCP), and a host and port. In our example, the server is willing to accept incoming connection requests on two different ports. It then waits for incoming messages. The request-reply pattern effectively ties the receipt of a message to the subsequent response. In other words, when the server calls send, it will transmit a message to the same client from which it previously had received a message. Of course, this simplicity can be achieved only if the programmer indeed abides to the request-reply pattern.

```

1 import zmq
2 context = zmq.Context()
3
4 p1 = "tcp://" + HOST + ":" + PORT1 # how and where to connect
5 p2 = "tcp://" + HOST + ":" + PORT2 # how and where to connect
6 s = context.socket(zmq.REP)      # create reply socket
7
8 s.bind(p1)                       # bind socket to address
9 s.bind(p2)                       # bind socket to address
10 while True:
11     message = s.recv()           # wait for incoming message
12     if not "STOP" in message:    # if not to stop...
13         s.send(message + "*")    # append "*" to message
14     else:                        # else...
15         break                   # break out of loop and end

```

Figure 4.22: (a) A ZeroMQ client-server system based: the server.

The client, shown in Figure 4.22(b) does what is expected: it creates a socket and connects to the associated server. When it sends a message, it can expect to

receive, from that same server, a response. By sending the string “STOP”, it tells the server it is done, after which the server will actually stop.

```

1 import zmq
2 context = zmq.Context()
3
4 p1 = "tcp://"+ HOST +":"+ PORT1 # how and where to connect
5 s = context.socket(zmq.REQ)      # create request socket
6
7 s.connect(p1)                   # block until connected
8 s.send("Hello world 1")         # send message
9 message = s.recv()              # block until response
10 s.send("STOP")                 # tell server to stop
11 print message                  # print result

```

Figure 4.22: (b) A ZeroMQ client-server system: the client.

Interestingly, the asynchronous nature of ZeroMQ allows one to start the client *before* starting the server. An implication is that if, in this example, we would start the server, then a client, and after a while a second client, that the latter will be blocked until the server is restarted. Furthermore, note that ZeroMQ does not require the programmer to specify how many bytes are expected to be received. Unlike TCP, ZeroMQ uses messages instead of bytestreams as its underlying model of communication.

In the case of a **publish-subscribe pattern**, clients **subscribe** to specific messages that are **published** by servers. We came across this pattern briefly in Chapter 1 when discussing event-based coordination. In effect, only the messages to which the client has subscribed will be transmitted. If a server is publishing messages to which no one has subscribed, these messages will be lost. In its simplest form, this pattern establishes multicasting messages from a server to several clients. The server is assumed to use a socket of type PUB, while each client must use SUB type sockets. Each client socket is connected to the socket of the server. By default, a client subscribes to no specific message. This means that as long as no explicit subscription is provided, a client will not receive a message published by the server.

Note 4.10 (Example: The publish-subscribe pattern)

Again, let us make this pattern more concrete through a simple example. Figure 4.23(a) shows an admittedly naive time server that publishes its current, local time, through a PUB socket. The local time is published every five seconds, for any interested client.

A client is equally simple, as shown in Figure 4.23(b). It first creates a SUB socket which it connects to the corresponding PUB socket of the server. In order to receive the appropriate messages, it needs to subscribe to messages that have TIME as their tag. In our example, a client will simply print the first five messages

received from the server. Note that we can have as many clients as we want: the server's message will be multicast to all subscribers.

```

1 import zmq, time
2
3 context = zmq.Context()
4 s = context.socket(zmq.PUB)      # create a publisher socket
5 p = "tcp://"+ HOST +":"+ PORT   # how and where to communicate
6 s.bind(p)                       # bind socket to the address
7 while True:
8     time.sleep(5)                # wait every 5 seconds
9     s.send("TIME " + time.asctime()) # publish the current time

```

Figure 4.23: (a) A multicasting socket-based time server

```

1 import zmq
2
3 context = zmq.Context()
4 s = context.socket(zmq.SUB)      # create a subscriber socket
5 p = "tcp://"+ HOST +":"+ PORT   # how and where to communicate
6 s.connect(p)                    # connect to the server
7 s.setsockopt(zmq.SUBSCRIBE, "TIME") # subscribe to TIME messages
8
9 for i in range(5): # Five iterations
10     time = s.recv() # receive a message
11     print time

```

Figure 4.23: (b) A client for the multicasting socket-based time server.

Finally, the **pipeline pattern** is characterized by the fact that a process wants to **push out** its results, assuming that there are other processes that want to **pull in** those results. The essence of the pipeline pattern is that a pushing process does not really care which other process pulls in its results: the first available one will do just fine. Likewise, any process pulling in results from multiple other processes will do so from the first pushing process making its results available. The intention of the pipeline pattern is thus seen to keep as many processes working as possible, pushing results through a pipeline of processes as quickly as possible.

Note 4.11 (Example: The pipeline pattern)

As our last example, consider the following template for keeping a collection of worker tasks busy. Figure 4.24(a) shows the code for a so-called **farmer task**: a process generating tasks to be picked up by others. In this example, we simulate the task by letting the farmer pick a random number modeling the duration, or load, of the work to be done. This workload is then sent to the push socket, effectively being queued until another process picks it up.


```

1 import zmq, time, pickle, sys, random
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 s = context.socket(zmq.PUSH)      # create a push socket
6 src = SRC1 if me == '1' else SRC2  # check task source host
7 prt = PORT1 if me == '1' else PORT2 # check task source port
8 p = "tcp://" + src + ":" + prt     # how and where to connect
9 s.bind(p)                         # bind socket to address
10
11 for i in range(100):               # generate 100 workloads
12     workload = random.randint(1, 100) # compute workload
13     s.send(pickle.dumps((me, workload))) # send workload to worker

```

Figure 4.24: (a) A task simulating the generation of work.

```

1 import zmq, time, pickle, sys
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 r = context.socket(zmq.PULL)      # create a pull socket
6 p1 = "tcp://" + SRC1 + ":" + PORT1 # address first task source
7 p2 = "tcp://" + SRC2 + ":" + PORT2 # address second task source
8 r.connect(p1)                    # connect to task source 1
9 r.connect(p2)                    # connect to task source 2
10
11 while True:
12     work = pickle.loads(r.recv()) # receive work from a source
13     time.sleep(work[1]*0.01)      # pretend to work

```

Figure 4.24: (b) A worker task.

Such other processes are known as **worker tasks**, of which a sketch is given in Figure 4.24(b). A worker task connects a single pull socket to, in this case, two specific farmer tasks. Once it picks up some work, it simulates that it is actually doing something by sleeping for a number of tens of milliseconds proportional to the received workload.

The semantics of this push-pull pattern is such that the first available worker will pick up work from either farmer, and likewise, if there are multiple workers ready to pick up work, each one of them will be provided with a task. How work distribution is actually done in a fair manner requires some specific attention which we will not discuss further here.

The Message-Passing Interface (MPI)

With the advent of high-performance multicomputers, developers have been looking for message-oriented operations that would allow them to easily write highly efficient applications. This means that the operations should be at a

convenient level of abstraction (to ease application development), and that their implementation incurs only minimal overhead. Sockets were deemed insufficient for two reasons. First, they were at the wrong level of abstraction by supporting only simple send and receive operations. Second, sockets had been designed to communicate across networks using general-purpose protocol stacks such as TCP/IP. They were not considered suitable for the proprietary protocols developed for high-speed interconnection networks, such as those used in high-performance server clusters. Those protocols required an interface that could handle more advanced features, such as different forms of buffering and synchronization.

The result was that most interconnection networks and high-performance multicomputers were shipped with proprietary communication libraries. These libraries offered a wealth of high-level and generally efficient communication operations. Of course, all libraries were mutually incompatible, so that application developers now had a portability problem.

The need to be hardware and platform independent eventually lead to the definition of a standard for message passing, simply called the **Message-Passing Interface** or **MPI**. MPI is designed for parallel applications and as such is tailored to transient communication. It makes direct use of the underlying network. Also, it assumes that serious failures such as process crashes or network partitions are fatal and do not require automatic recovery.

MPI assumes communication takes place within a known group of processes. Each group is assigned an identifier. Each process within a group is also assigned a (local) identifier. A (groupID, processID) pair therefore uniquely identifies the source or destination of a message, and is used instead of a transport-level address. There may be several, possibly overlapping groups of processes involved in a computation and that are all executing at the same time.

At the core of MPI are messaging operations to support transient communication, of which the most intuitive ones are summarized in Figure 4.25.

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_issend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Figure 4.25: Some of the most intuitive message-passing operations of MPI.

Transient asynchronous communication is supported by means of the `MPI_bsend` operation. The sender submits a message for transmission, which is generally first copied to a local buffer in the MPI runtime system. When the message has been copied, the sender continues. The local MPI runtime system will remove the message from its local buffer and take care of transmission as soon as a receiver has called a receive operation.

There is also a blocking send operation, called `MPI_send`, of which the semantics are implementation dependent. The operation `MPI_send` may either block the caller until the specified message has been copied to the MPI runtime system at the sender's side, or until the receiver has initiated a receive operation. Synchronous communication by which the sender blocks until its request is accepted for further processing is available through the `MPI_ssend` operation. Finally, the strongest form of synchronous communication is also supported: when a sender calls `MPI_sendrecv`, it sends a request to the receiver and blocks until the latter returns a reply. Basically, this operation corresponds to a normal RPC.

Both `MPI_send` and `MPI_ssend` have variants that avoid copying messages from user buffers to buffers internal to the local MPI runtime system. These variants essentially correspond to a form of asynchronous communication. With `MPI_isend`, a sender passes a pointer to the message after which the MPI runtime system takes care of communication. The sender immediately continues. To prevent overwriting the message before communication completes, MPI offers operations to check for completion, or even to block if required. As with `MPI_send`, whether the message has actually been transferred to the receiver or that it has merely been copied by the local MPI runtime system to an internal buffer is left unspecified.

Likewise, with `MPI_issend`, a sender also passes only a pointer to the MPI runtime system. When the runtime system indicates it has processed the message, the sender is then guaranteed that the receiver has accepted the message and is now working on it.

The operation `MPI_recv` is called to receive a message; it blocks the caller until a message arrives. There is also an asynchronous variant, called `MPI_irecv`, by which a receiver indicates that it is prepared to accept a message. The receiver can check whether or not a message has indeed arrived, or block until one does.

The semantics of MPI communication operations are not always straightforward, and different operations can sometimes be interchanged without affecting the correctness of a program. The official reason why so many different forms of communication are supported is that it gives implementers of MPI systems enough possibilities for optimizing performance. Cynics might say the committee could not make up its collective mind, so it threw in everything. By now, MPI is in its third version with over 440 operations available. Being designed for high-performance parallel applications, it is perhaps easier

to understand its diversity. More on MPI can be found in [Gropp et al., 2016]. The complete MPI-3 reference can be found in [MPI Forum, 2012].

Message-oriented persistent communication

We now come to an important class of message-oriented middleware services, generally known as **message-queuing systems**, or just **Message-Oriented Middleware (MOM)**. Message-queuing systems provide extensive support for persistent asynchronous communication. The essence of these systems is that they offer intermediate-term storage capacity for messages, without requiring either the sender or receiver to be active during message transmission. An important difference with sockets and MPI is that message-queuing systems are typically targeted to support message transfers that are allowed to take minutes instead of seconds or milliseconds.

Message-queuing model

The basic idea behind a message-queuing system is that applications communicate by inserting messages in specific queues. These messages are forwarded over a series of communication servers and are eventually delivered to the destination, even if it was down when the message was sent. In practice, most communication servers are directly connected to each other. In other words, a message is generally transferred directly to a destination server. In principle, each application has its own private queue to which other applications can send messages. A queue can be read only by its associated application, but it is also possible for multiple applications to share a single queue.

An important aspect of message-queuing systems is that a sender is generally given only the guarantees that its message will eventually be inserted in the recipient's queue. No guarantees are given about when, or even if the message will actually be read, which is completely determined by the behavior of the recipient.

These semantics permit communication to be loosely coupled in time. There is thus no need for the receiver to be executing when a message is being sent to its queue. Likewise, there is no need for the sender to be executing at the moment its message is picked up by the receiver. The sender and receiver can execute completely independently of each other. In fact, once a message has been deposited in a queue, it will remain there until it is removed, irrespective of whether its sender or receiver is executing. This gives us four combinations with respect to the execution mode of the sender and receiver, as shown in Figure 4.26.

In Figure 4.26(a), both the sender and receiver execute during the entire transmission of a message. In Figure 4.26(b), only the sender is executing, while the receiver is passive, that is, in a state in which message delivery is not possible. Nevertheless, the sender can still send messages. The combination of

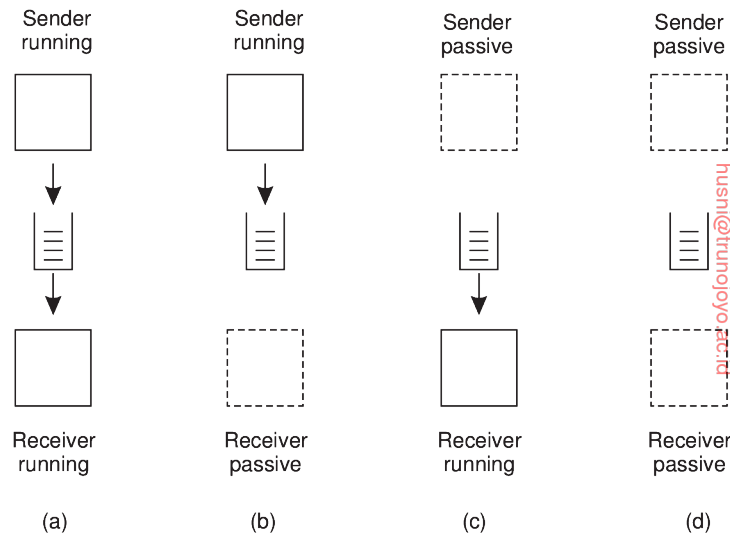


Figure 4.26: Four combinations for loosely-coupled communication using queues.

a passive sender and an executing receiver is shown in Figure 4.26(c). In this case, the receiver can read messages that were sent to it, but it is not necessary that their respective senders are executing as well. Finally, in Figure 4.26(d), we see the situation that the system is storing (and possibly transmitting) messages even while sender and receiver are passive. One may argue that only if this last configuration is supported, the message-queuing system truly provides persistent messaging.

Messages can, in principle, contain any data. The only important aspect from the perspective of middleware is that messages are properly addressed. In practice, addressing is done by providing a systemwide unique name of the destination queue. In some cases, message size may be limited, although it is also possible that the underlying system takes care of fragmenting and assembling large messages in a way that is completely transparent to applications. An effect of this approach is that the basic interface offered to applications can be extremely simple, as shown in Figure 4.27.

The put operation is called by a sender to pass a message to the underlying system that is to be appended to the specified queue. As we explained, this is a nonblocking call. The get operation is a blocking call by which an authorized process can remove the longest pending message in the specified queue. The process is blocked only if the queue is empty. Variations on this call allow searching for a specific message in the queue, for example, using a priority, or a matching pattern. The nonblocking variant is given by the poll operation. If the queue is empty, or if a specific message could not be found, the calling

Operation	Description
put	Append a message to a specified queue
get	Block until the specified queue is nonempty, and remove the first message
poll	Check a specified queue for messages, and remove the first. Never block
notify	Install a handler to be called when a message is put into the specified queue

Figure 4.27: Basic interface to a queue in a message-queuing system.

process simply continues.

Finally, most queuing systems also allow a process to install a handler as a *callback function*, which is automatically invoked whenever a message is put into the queue. Callbacks can also be used to automatically start a process that will fetch messages from the queue if no process is currently executing. This approach is often implemented by means of a daemon on the receiver's side that continuously monitors the queue for incoming messages and handles accordingly.

General architecture of a message-queuing system

Let us now take a closer look at what a general message-queuing system looks like. First of all, queues are managed by **queue managers**. A queue manager is either a separate process, or is implemented by means of a library that is linked with an application. Secondly, as a rule of thumb, an application can put messages only into a *local* queue. Likewise, getting a message is possible by extracting it from a *local* queue only. As a consequence, if a queue manager QM_A handling the queues for an application A runs as a separate process, both processes QM_A and A will generally be placed on the same machine, or at worst on the same LAN. Also note that if *all* queue managers are linked into their respective applications, we can no longer speak of a persistent asynchronous messaging system.

If applications can put messages only into local queues, then clearly each message will have to carry information concerning its destination. It is the queue manager's task to make sure that a message reaches its destination. This brings us to a number of issues.

In the first place, we need to consider how the destination queue is addressed. Obviously, to enhance location transparency, it is preferable that queues have logical, location-independent names. Assuming that a queue manager is implemented as a separate process, using logical names implies that each name should be associated with a **contact address**, such as a (host,port)-pair, and that the name-to-address mapping is readily available to a queue

manager, as shown in Figure 4.28. In practice, a contact address carries more information, notably the protocol to be used, such as TCP or UDP. We came across such contact addresses in our examples of advanced sockets in, for example, Note 4.9.

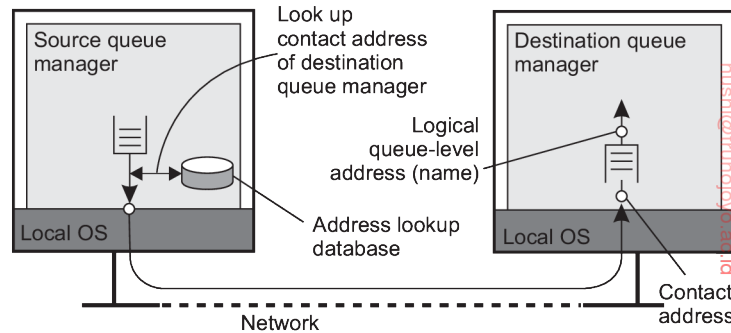


Figure 4.28: The relationship between queue-level naming and network-level addressing.

A second issue that we need to consider is how the name-to-address mapping is actually made available to a queue manager. A common approach is to simply implement the mapping as a lookup table and copy that table to all managers. Obviously, this leads to a maintenance problem, for every time that a new queue is added or named, many, if not all tables need to be updated. There are various ways to alleviate such problems, which we will discuss in Chapter 5.

This brings us to a third issue, related to the problems of efficiently maintaining name-to-address mappings. We have implicitly assumed that if a destination queue at manager QM_B is known to queue manager QM_A , then QM_A can directly contact QM_B to transfer messages. In effect, this means that (the contact address of) each queue manager should be known to all others. Obviously, when dealing with very large message-queuing systems, we will have a scalability problem. In practice, there are often special queue managers that operate as **routers**: they forward incoming messages to other queue managers. In this way, a message-queuing system may gradually grow into a complete, application-level, **overlay network**.

If only a few routers need to know about the network topology, then a source queue manager need only to know to which adjacent router, say R it should forward a message, given a destination queue. Router R , in turn, may only need to keep track of its adjacent routers to see where to forward the message to, and so on. Of course, we still need to have name-to-address mappings for all queue managers, including the routers, but it is not difficult to imagine that such tables can be much smaller and easier to maintain.

Message brokers

An important application area of message-queuing systems is integrating existing and new applications into a single, coherent distributed information system. If we assume that communication with an application takes place through messages, then integration requires that applications can understand the messages they receive. In practice, this requires the sender to have its outgoing messages in the same format as that of the receiver, but also that its messages adhere to the same semantics as those expected by the receiver. Sender and receiver essentially need to speak the same language, that is, adhere to the same messaging protocol.

The problem with this approach is that each time an application A is added to the system having its own messaging protocol, then for each other application B that is to communicate with A we will need to provide the means for converting their respective messages. In a system with N applications, we will thus need $N \times N$ messaging protocol converters.

An alternative is to agree on a common messaging protocol, as is done with traditional network protocols. Unfortunately, this approach will generally not work for message-queuing systems. The problem is the level of abstraction at which these systems operate. A common messaging protocol makes sense only if the collection of processes that make use of that protocol indeed have enough in common. If the collection of applications that make up a distributed information system is highly diverse (which it often is), then inventing a one-size-fits-all solution is simply not going to work.

If we focus only on the format and meaning of messages, commonality can be achieved by lifting the level of abstraction as is done with XML messages. In this case, messages carry information on their own organization, and what has been standardized is the way that they can describe their content. As a consequence, an application can provide information on the organization of its messages that can be automatically processed. Of course, this information is generally not enough: we also need to make sure that the *semantics* of messages are well understood.

Given these problems, the general approach is to learn to live with differences, and try to provide the means to make conversions as simple as possible. In message-queuing systems, conversions are handled by special nodes in a queuing network, known as **message brokers**. A message broker acts as an application-level gateway in a message-queuing system. Its main purpose is to convert incoming messages so that they can be understood by the destination application. Note that to a message-queuing system, a message broker is just another application, as shown in Figure 4.29. In other words, a message broker is generally not considered to be an integral part of the queuing system.

A message broker can be as simple as a reformatter for messages. For example, assume an incoming message contains a table from a database in which records are separated by a special *end-of-record* delimiter and fields

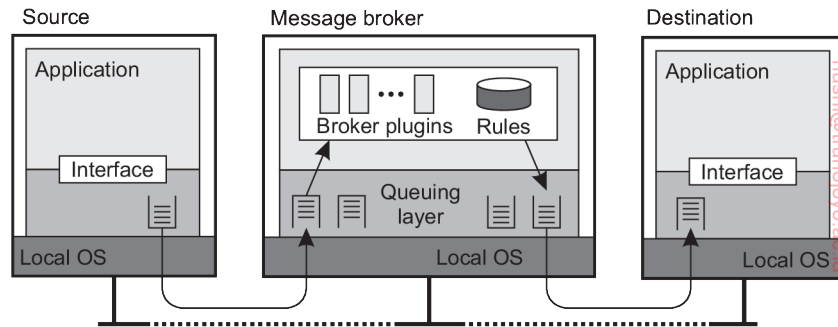


Figure 4.29: The general organization of a message broker in a message-queuing system.

within a record have a known, fixed length. If the destination application expects a different delimiter between records, and also expects that fields have variable lengths, a message broker can be used to convert messages to the format expected by the destination.

In a more advanced setting, a message broker may act as an application-level gateway, in which information on the messaging protocol of several applications has been encoded. In general, for each pair of applications, we will have a separate subprogram capable of converting messages between the two applications. In Figure 4.29, this subprogram is drawn as a **plugin** to emphasize that such parts can be dynamically plugged in, or removed from a broker.

Finally, note that in many cases a message broker is used for advanced **enterprise application integration (EAI)**, as we discussed in Section 1.3. In this case, rather than (only) converting messages, a broker is responsible for *matching* applications based on the messages that are being exchanged. In such a **publish-subscribe** model, applications send messages in the form of *publishing*. In particular, they may publish a message on topic X, which is then sent to the broker. Applications that have stated their interest in messages on topic X, that is, who have *subscribed* to those messages, will then receive these messages from the broker. More advanced forms of mediation are also possible.

At the heart of a message broker lies a repository of rules for transforming a message of one type to another. The problem is defining the rules and developing the plugins. Most message broker products come with sophisticated development tools, but the bottom line is still that the repository needs to be filled by experts. Here we see a perfect example where commercial products are often misleadingly said to provide “intelligence,” where, in fact, the only intelligence is to be found in the heads of those experts.

Note 4.12 (More information: A note on message-queuing systems)

Considering what we have said about message-queuing systems, it would appear that they have long existed in the form of implementations for e-mail services. E-mail systems are generally implemented through a collection of mail servers that store and forward messages on behalf of the users on hosts directly connected to the server. Routing is generally left out, as e-mail systems can make direct use of the underlying transport services. For example, in the mail protocol for the Internet, SMTP [Postel, 1982], a message is transferred by setting up a direct TCP connection to the destination mail server.

What makes e-mail systems special compared to message-queuing systems is that they are primarily aimed at providing direct support for end users. This explains, for example, why a number of groupware applications are based directly on an e-mail system [Khoshafian and Buckiewicz, 1995]. In addition, e-mail systems may have very specific requirements such as automatic message filtering, support for advanced messaging databases (e.g., to easily retrieve previously stored messages), and so on.

General message-queuing systems are not aimed at supporting only end users. An important issue is that they are set up to enable persistent communication between processes, regardless of whether a process is running a user application, handling access to a database, performing computations, and so on. This approach leads to a different set of requirements for message-queuing systems than pure e-mail systems. For example, e-mail systems generally need not provide guaranteed message delivery, message priorities, logging facilities, efficient multicasting, load balancing, fault tolerance, and so on for general usage.

General-purpose message-queuing systems, therefore, have a wide range of applications, including e-mail, workflow, groupware, and batch processing. However, as we have stated before, the most important application area is the integration of a (possibly widely-dispersed) collection of databases and applications into a federated information system [Hohpe and Woolf, 2004]. For example, a query expanding several databases may need to be split into subqueries that are forwarded to individual databases. Message-queuing systems assist by providing the basic means to package each subquery into a message and routing it to the appropriate database. Other communication facilities we have discussed in this chapter are far less appropriate.

Example: IBM's WebSphere message-queuing system

To help understand how message-queuing systems work in practice, let us take a look at the message-queuing system that is part of IBM's WebSphere. Formerly known as MQSeries, it is now referred to as **WebSphere MQ**. There is a wealth of documentation on WebSphere MQ, and in the following we can only resort to the basic principles. Taylor [2012] provides a gentle introduction to WebSphere MQ, whereas further details can be found in [Davies and Broadhurst, 2005; Aranha et al., 2013].

Overview

The basic architecture of an MQ queuing network is quite straightforward, and is shown in Figure 4.30. All queues are managed by **queue managers**. A queue manager is responsible for removing messages from its send queues, and forwarding those to other queue managers. Likewise, a queue manager is responsible for handling incoming messages by picking them up from the underlying network and subsequently storing each message in the appropriate input queue. To give an impression of what messaging can mean: a message has a maximum default size of 4 MB, but this can be increased up to 100 MB. A queue is normally restricted to 2 GB of data, but depending on the underlying operating system, this maximum can be easily set higher.

Queue managers are pairwise connected through **message channels**, which are an abstraction of transport-level connections. A message channel is a uni-directional, reliable connection between a sending and a receiving queue manager, through which queued messages are transported. For example, an Internet-based message channel is implemented as a TCP connection. Each of the two ends of a message channel is managed by a **message channel agent (MCA)**. A sending MCA is basically doing nothing else than checking send queues for a message, wrapping it into a transport-level packet, and sending it along the connection to its associated receiving MCA. Likewise, the basic task of a receiving MCA is listening for an incoming packet, unwrapping it, and subsequently storing the unwrapped message into the appropriate queue.

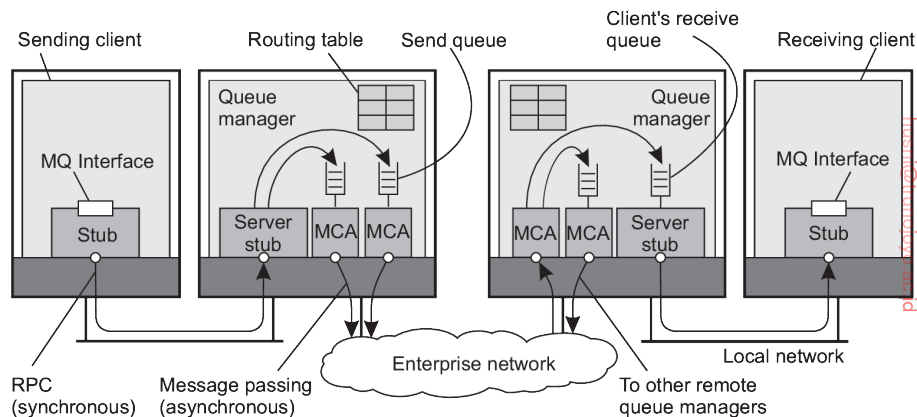


Figure 4.30: General organization of IBM's message-queuing system.

Queue managers can be linked into the same process as the application for which it manages the queues. In that case, the queues are hidden from the application behind a standard interface, but effectively can be directly manipulated by the application. An alternative organization is one in which queue managers and applications run on separate machines. In that case,

the application is offered the same interface as when the queue manager is co-located on the same machine. However, the interface is implemented as a proxy that communicates with the queue manager using traditional RPC-based synchronous communication. In this way, MQ basically retains the model that only queues local to an application can be accessed.

Channels

An important component of MQ is formed by the message channels. Each message channel has exactly one associated send queue from which it fetches the messages it should transfer to the other end. Transfer along the channel can take place only if both its sending and receiving MCA are up and running. Apart from starting both MCAs manually, there are several alternative ways to start a channel, some of which we discuss next.

One alternative is to have an application directly start its end of a channel by activating the sending or receiving MCA. However, from a transparency point of view, this is not a very attractive alternative. A better approach to start a *sending* MCA is to configure the channel's send queue to set off a trigger when a message is first put into the queue. That trigger is associated with a handler to start the sending MCA so that it can remove messages from the send queue.

Another alternative is to start an MCA over the network. In particular, if one side of a channel is already active, it can send a control message requesting that the other MCA is to be started. Such a control message is sent to a daemon listening to a well-known address on the same machine as where the other MCA is to be started.

Channels are stopped automatically after a specified time has expired during which no more messages were dropped into the send queue.

Each MCA has a set of associated attributes that determine the overall behavior of a channel. Some of the attributes are listed in Figure 4.31. Attribute values of the sending and receiving MCA should be compatible and perhaps negotiated first before a channel can be set up. For example, both MCAs should obviously support the same transport protocol. An example of a nonnegotiable attribute is whether or not messages are to be delivered in the same order as they are put into the send queue. If one MCA wants FIFO delivery, the other must comply. An example of a negotiable attribute value is the maximum message length, which will simply be chosen as the minimum value specified by either MCA.

Message transfer

To transfer a message from one queue manager to another (possibly remote) queue manager, it is necessary that each message carries its destination address, for which a transmission header is used. An address in MQ consists of

Attribute	Description
Transport type	Determines the transport protocol to be used
FIFO delivery	Indicates that messages are to be delivered in the order they are sent
Message length	Maximum length of a single message
Setup retry count	Maximum number of retries to start up the remote MCA
Delivery retries	Maximum times MCA will try to put received message into queue

Figure 4.31: Some attributes associated with message channel agents.

two parts. The first part consists of the name of the queue manager to which the message is to be delivered. The second part is the name of the destination queue resorting under that manager to which the message is to be appended.

Besides the destination address, it is also necessary to specify the route that a message should follow. Route specification is done by providing the name of the local send queue to which a message is to be appended. Thus it is not necessary to provide the full route in a message. Recall that each message channel has exactly one send queue. By telling to which send queue a message is to be appended, we effectively specify to which queue manager a message is to be forwarded.

In most cases, routes are explicitly stored inside a queue manager in a routing table. An entry in a routing table is a pair (destQM, sendQ), where destQM is the name of the destination queue manager, and sendQ is the name of the local send queue to which a message for that queue manager should be appended. (A routing table entry is called an alias in MQ.)

It is possible that a message needs to be transferred across multiple queue managers before reaching its destination. Whenever such an intermediate queue manager receives the message, it simply extracts the name of the destination queue manager from the message header, and does a routing-table look-up to find the local send queue to which the message should be appended.

It is important to realize that each queue manager has a systemwide unique name that is effectively used as an identifier for that queue manager. The problem with using these names is that replacing a queue manager, or changing its name, will affect all applications that send messages to it. Problems can be alleviated by using a **local alias** for queue manager names. An alias defined within a queue manager QM₁ is another name for a queue manager QM₂, but which is available only to applications interfacing to QM₁. An alias allows the use of the same (logical) name for a queue, even if the queue manager of that queue changes. Changing the name of a queue manager requires that we change its alias in all queue managers. However, applications can be left unaffected.

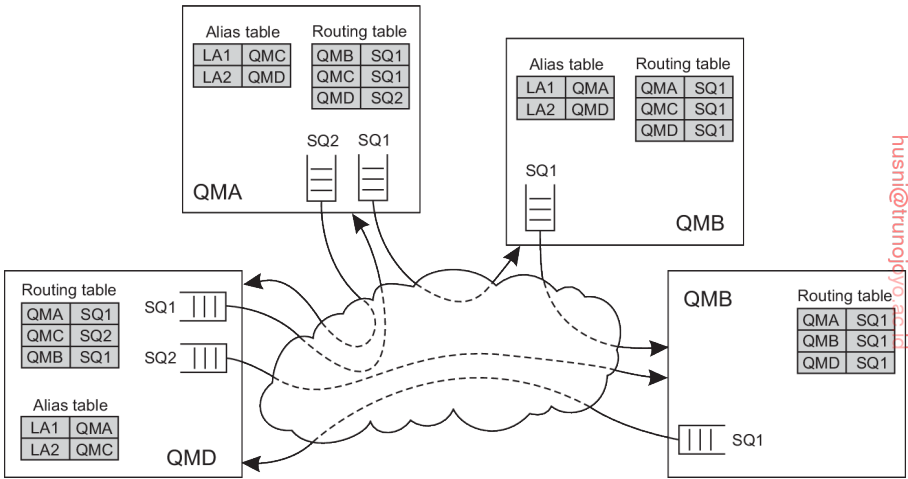


Figure 4.32: The general organization of an MQ queuing network using routing tables and aliases.

The principle of using routing tables and aliases is shown in Figure 4.32. For example, an application linked to queue manager QMA can refer to a remote queue manager using the local alias LA₁. The queue manager will first look up the actual destination in the alias table to find it is queue manager QMC. The route to QMC is found in the routing table, which states that messages for QMC should be appended to the outgoing queue SQ₁, which is used to transfer messages to queue manager QMB. The latter will use its routing table to forward the message to QMC.

Following this approach of routing and aliasing leads to a programming interface that is relatively simple, called the **Message Queue Interface (MQI)**. The most important operations of MQI are summarized in Figure 4.33.

Operation	Description
MQOPEN	Open a (possibly remote) queue
MQCLOSE	Close a queue
MQPUT	Put a message into an opened queue
MQGET	Get a message from a (local) queue

Figure 4.33: Operations available in the message-queuing interface.

To put messages into a queue, an application calls MQOPEN, specifying a destination queue in a specific queue manager. The queue manager can be named using the locally-available alias. Whether the destination queue is actually remote or not is completely transparent to the application. MQOPEN should also be called if the application wants to get messages from its local

queue. Only local queues can be opened for reading incoming messages. When an application is finished with accessing a queue, it should close it by calling `MQCLOSE`.

Messages can be written to, or read from, a queue using `MQPUT` and `MQGET`, respectively. In principle, messages are removed from a queue on a priority basis. Messages with the same priority are removed on a first-in, first-out basis, that is, the longest pending message is removed first. It is also possible to request for specific messages. Furthermore, MQ provides facilities to signal applications when messages have arrived, thus avoiding that an application will continuously have to poll a message queue for incoming messages.

An interesting observation, and one that is common to modern message-queuing systems, is that *persistent* messaging is not obtained by simply making queues persistent. Instead, a message is marked as being persistent, and it is a queue manager's job to see to it that the message can survive a crash. As a consequence, a queue can simultaneously store persistent and nonpersistent messages.

Managing overlay networks

From the description so far, it should be clear that an important part of managing MQ systems is connecting the various queue managers into a consistent overlay network. Moreover, this network needs to be maintained over time. For small networks, this maintenance will not require much more than average administrative work, but matters become complicated when message queuing is used to integrate and disintegrate large existing systems.

A major issue with MQ is that overlay networks need to be manually administrated. This administration not only involves creating channels between queue managers, but also filling in the routing tables. Obviously, this can grow into a nightmare. Unfortunately, management support for MQ systems is advanced only in the sense that an administrator can set virtually every possible attribute, and tweak any thinkable configuration. However, the bottom line is that channels and routing tables need to be manually maintained.

At the heart of overlay management is the **channel control function** component, which logically sits between message channel agents. This component allows an operator to monitor exactly what is going on at two end points of a channel. In addition, it is used to create channels and routing tables, but also to manage the queue managers that host the message channel agents. In a way, this approach to overlay management strongly resembles the management of cluster servers where a single administration server is used. In the latter case, the server essentially offers only a remote shell to each machine in the cluster, along with a few collective operations to handle groups of machines. The good news about distributed-systems management is that it offers lots of opportunities if you are looking for an area to explore new solutions to serious problems.

Example: Advanced Message Queuing Protocol (AMQP)

An interesting observation about message-queuing systems is that they have been developed in part to allow legacy applications to interoperate, yet at the same time we see that when it comes to operations between different message-queuing systems, we often hit a wall. As a consequence, once an organization chooses to use a message-queuing system from manufacturer X, they may have to settle for solutions that only X provides. Message-queuing solutions are thus in large part proprietary solutions. So much for openness.

In 2006, a working group was formed to change this situation, which resulted in the specification of the **Advanced Message-Queuing Protocol**, or simply **AMQP**. There are different versions of AMQP, with version 1.0 being the most recent one. There are also various implementations of AMQP, notably of versions prior to 1.0, which by the time version 1.0 was established, had gained considerable popularity. Because a pre-1.0 version is so different from the 1.0 version, yet has also a steady user base, we may see various pre-1.0 AMQP servers exist next to (their undeniably incompatible) 1.0 servers. So much for openness.

In this section, we will describe AMQP, but will more or less deliberately mix the pre-1.0 and 1.0 versions, sticking to the essentials and spirit of AMQP. Details can be found in the specifications [Group, 2008; OASIS, 2011]. Implementations of AMQP include RabbitMQ [Videla and Williams, 2012] and Apache's Qpid.

Basics

AMQP evolves around applications, queue managers, and queues. Taking an approach that is common for many networking situations, we make a distinction between AMQP as a *messaging service*, the actual *messaging protocol*, and, finally, the *messaging interface* as offered to applications. To this end, it is easiest to consider the situation of having only a single queue manager, running as a single, separate server forming the implementation of AMQP as a service. An application communicates with this queue manager through a local interface. Between an application and the queue manager communication proceeds according to the AMQP protocol.

This situation is shown in Figure 4.34 and should look familiar. The AMQP stub shields the application (as well as the queue manager) from the details concerning message transfer and communication in general. At the same time, it implements a message-queuing interface, allowing the application to make use of AMQP as a message-queuing service. Although the distinction between AMQP stub and queue manager is made explicit for queue managers, the strictness of the separation is left to an implementation. Nevertheless, if not strict, conceptually there is a distinction between handling queues and handling related communication as we shall make clear shortly.

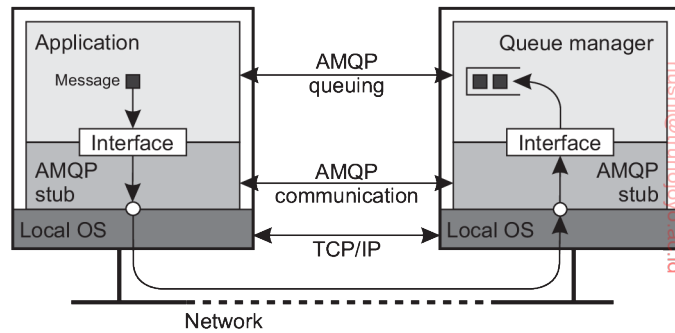


Figure 4.34: An overview of a single-server AMQP instance.

AMQP communication

AMQP allows an application to set up a **connection** to a queue manager; a connection is a container for a number of one-way **channels**. Whereas the lifetime of a channel can be highly dynamic, connections are assumed to be relatively stable. This difference between connection and channel allows for efficient implementations, notably by using a single transport-layer TCP connection to multiplex lots of different channels between an application and a queue manager. In practice, AMQP assumes TCP is used for establishing AMQP connections.

Bidirectional communication is established through **sessions**: a logical grouping of two channels. A connection may have multiple sessions, but note that a channel need not necessarily be part of a session.

Finally, to actually transfer messages, a **link** is needed. Conceptually, a link, or rather its end points, keep track of the status of messages that are being transferred. It thus provides fine-grained flow control between an application and a queue manager, and, indeed, different control policies can be put simultaneously in place for different messages that are transferred through the same session of connection. Flow control is established through credits: a receiver can tell the sender how many messages it is allowed to send over a specific link.

When a message is to be transferred, the application passes it to its local AMQP stub. As mentioned, each message transfer is associated with one specific link. Message transfer normally proceeds in three steps.

1. At the sender's side, the message is assigned a unique identifier and is recorded *locally* to be in an **unsettled state**. The stub subsequently transfers the message to the server, where the AMQP stub also records it as being in an unsettled state. At that point, the server-side stub passes it to the queue manager.

2. The receiving application (in this case the queue manager), is assumed to handle the message and normally reports back to its stub that it is finished. The stub passes this information to the original sender, at which point the message at the original sender's AMQP stub enters a **settled state**.
3. The AMQP stub of the original sender now tells the stub of the original receiver that message transfer has been settled (meaning that the original sender will forget about the message from now on). The receiver's stub can now also discard anything about the message, formally recording it as being settled as well.

Note that because the receiving application can indicate to the underlying AMQP communication layer that it is done with a message, AMQP enables true end-to-end communication reliability. In particular, the application, be it a client application or an actual queue manager, can instruct the AMQP communication layer to keep hold of a message (i.e., a message stays in the unsettled state).

AMQP messaging

Messaging in AMQP logically takes place at the layer above the one handling communication. It is here that an application can indicate what needs to be done with a message, but can also see what has happened so far. Messaging formally takes place between two **nodes**, of which there are three types: a producer, a consumer, or a queue. Typically, producer and consumer nodes represent regular applications, whereas queues are used to store and forward messages. Indeed, a queue manager will typically consist of multiple queue nodes. In order for message transfer to take place, two nodes will have to establish a link between them.

The receiver can indicate to the sender whether its message was accepted (meaning that it was successfully processed), or rejected. Note that this means that a notification is returned to the original sender. AMQP also supports fragmentation and assembly of large messages for which additional notifications are sent.

Of course, an important aspect of AMQP is its support for *persistent* messaging. Achieving persistence is handled through several mechanisms. First and foremost important, a message can be marked as durable, indicating that the source expects any intermediate node, such as a queue, to be able to recover in the case of a failure. An intermediate node that cannot guarantee such durability will have to reject a message. Second, a source or target node can also indicate its durability: if durable, will it maintain its state, or will it also maintain the (unsettled) state of durable messages? Combining the latter with durable messages effectively establishes reliable message transfer and persistent messaging.

AMQP is truly a messaging protocol in the sense that it does not by itself support, for example, publish-subscribe primitives. It expects that such issues are handled by more advanced, proprietary queue managers, akin to the message brokers discussed in Section 4.3.

Finally, there is no reason why a queue manager cannot be connected to another queue manager. In fact, it is quite common to organize queue managers into an overlay network in which messages are routed from producers to their consumers. AMQP does not specify how the overlay network should be constructed and managed, and, indeed, different providers of AMQP-based systems offer different solutions. Of particular importance is specifying how messages should be routed through the network. The bottom line is that administrators will need to do a lot of this specification manually. Only in cases where overlays have regular structures, such as cycles or trees, it becomes easier to provide the necessary routing details.

4.4 Multicast communication

An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as multicast communication. For many years, this topic has belonged to the domain of network protocols, where numerous proposals for network-level and transport-level solutions have been implemented and evaluated [Janic, 2005; Obraczka, 1998]. A major issue in all solutions was setting up the communication paths for information dissemination. In practice, this involved a huge management effort, in many cases requiring human intervention. In addition, as long as there is no convergence of proposals, ISPs have shown to be reluctant to support multicasting [Diot et al., 2000].

With the advent of peer-to-peer technology, and notably structured overlay management, it became easier to set up communication paths. As peer-to-peer solutions are typically deployed at the application layer, various application-level multicasting techniques have been introduced. In this section, we will take a brief look at these techniques.

Multicast communication can also be accomplished in other ways than setting up explicit communication paths. As we also explore in this section, gossip-based information dissemination provides simple (yet often less efficient) ways for multicasting.

Application-level tree-based multicasting

The basic idea in application-level multicasting is that nodes organize into an overlay network, which is then used to disseminate information to its members. An important observation is that network routers are not involved in group membership. As a consequence, the connections between nodes in the overlay network may cross several physical links, and as such, routing